

Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



**Valutazione di pgvector come
database unificato per RAG**

Tesi di laurea triennale

Relatore

Marco Zanella

Laureando

Davide Lorenzon

Matricola 2101075

I hate perfection. To be perfect is to be unable to improve any further.

— Kurotsuchi Mayuri

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella relatore della mia tesi, per l'aiuto e il continuo sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare con affetto i miei genitori e i miei parenti per il sostegno e per essermi stati vicini durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici per tutti i bellissimi anni passati insieme.

Ci tengo infine a ringraziare i colleghi di Pat SRL e il tutor aziendale Davide Bastianetto per avermi dato l'opportunità di lavorare a questo progetto.

Padova, Settembre 2026

Davide Lorenzon

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di circa trecentoventi ore, dal laureando Davide Lorenzon presso l'azienda Pat SRL. Lo stage è stato condotto sotto la supervisione del tutor aziendale Davide Bastianetto, mentre il prof. Marco Zanella ha ricoperto il ruolo di tutor accademico.

Al centro di questo elaborato vi è la progettazione e lo sviluppo del modulo di information retrieval basato su ricerca semantica, ricerca full-text e ricerca ibrida.

Questo sistema trova applicazione nella parte retrieval dei sistemi *RAG_G*, oggi fondamentale poiché permettono agli *LLM_G* di accedere a basi di conoscenza private e informazioni aggiornate senza la necessità di ricorrere a costosi processi di retraining, abbattendo drasticamente il rischio di allucinazioni da parte dell'Intelligenza Artificiale.

Lo scopo principale del progetto è valutare l'adeguatezza, la fattibilità tecnica e le performance dell'estensione *Pgvector_G* per Postgres, impiegandola come database unificato. Nello specifico, l'obiettivo è verificare se tale tecnologia possa supportare efficacemente l'indicizzazione dei documenti, la ricerca ibrida (combinazione di ricerca full-text e semantica), l'applicazione di strategie di ranking avanzate e la correlazione relazionale con entità strutturate, in particolare swi valuterà una modalità di ricerca detta linked che permette di cercare un'informazione all'interno dell'intero database.

Per convalidare questa ipotesi e fornire una misura rigorosa della qualità del sistema, l'infrastruttura progettata verrà sottoposta a test comparativi contro una pipeline basata su *Elasticsearch_G*, tecnologia attualmente adottata all'interno dell'impresa.

I 2 sistemi potrebbero non adottare approcci equivalenti qualora pg-vector e Postgres permettano di implementare funzionalità utili non attualmente utilizzate sul sistema basato su elasticsearch.

L'utilità e il valore aggiunto di questa ricerca risiedono nella potenziale semplificazione dell'infrastruttura IT aziendale. Gestire dati relazionali, testuali e vettoriali all'interno di un unico ecosistema permetterebbe di

superare il paradigma della persistenza poliglotta e dei workaround per implementare concetti relazionali come le relazioni tra entità, eliminando la necessità di mantenere sistemi separati. Questo approccio promette di abbattere l'overhead di sincronizzazione dei dati, ridurre i costi di manutenzione sistemistica e garantire transazioni più sicure.

Organizzazione del testo

- Il primo capitolo** introduce l'azienda, il progetto e le motivazioni che mi hanno portato a sceglierlo;
- Il secondo capitolo** descrive l'azienda, il progetto e l'organizzazione del lavoro, definendo gli obiettivi e analizzando i rischi;
- Il terzo capitolo** descrive l'analisi dei requisiti del progetto, indicando un'analisi degli utenti, i casi d'uso e il tracciamento dei requisiti;
- Il quarto capitolo** descrive le tecnologie esistenti per risolvere i problemi indicando gli aspetti teorici alla base, gli strumenti che sono stati scelti e con quali criteri; ,) [Il quarto capitolo]: descrive le tecnologie esistenti per risolvere i problemi indicando gli aspetti teorici alla base, gli strumenti che sono stati scelti e con quali criteri;
- Il quinto capitolo** descrive nel dettaglio le problematiche sorte nel concreto durante lo svolgimento del progetto;
- Il sesto capitolo** raggruppa le conclusioni tratte dallo svolgimento del progetto.

Convenzioni tipografiche

Durante la stesura del testo ho scelto di adottare le seguenti convenzioni tipografiche: Le convenzioni tipografiche sono momentaneamente sospese e saranno riprese durante la stesura finale della tesi

- I blocchi di codice sono rappresentati nel seguente modo:


```

1  float Q_rsqr( float number ){
2      long i;
3      float x2, y;
4      const float threehalfs = 1.5F;
5      x2 = number * 0.5F;
6      y = number;
7      i = * (long * ) &y;
8      i = 0x5f3759df - (i>>1);
9      y = * (float * ) &i;
10     y = y * ( threehalfs - ( x2 * y * y ) );
11     //y = y * ( threehalfs - ( x2 * y * y ) );
12     return y;
13 }

```

Codice 1: Codice d'esempio.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	1
1.3	Scelta del progetto	3
2	Descrizione stage	5
2.1	Competenze da apprendere	5
2.2	Vincoli	5
2.3	Pianificazione	6
2.4	Analisi dei rischi	6
3	Analisi dei requisiti	15
3.1	Analisi degli Utenti	15
3.1.1	Attori primari	15
3.1.2	Attori secondari	16
3.2	Casi d'uso	16
3.2.1	Struttura della scheda descrittiva	16
3.2.2	Lista dei casi d'Uso	18
3.3	Tracciamento dei requisiti	43
4	Introduzione Teorica	55
4.1	Contesto e problema	55
4.1.1	Limiti di Elasticsearch	56
4.2	Basi teoriche	56
4.3	Architettura del progetto	57
4.3.1	Sistema principale	58
4.3.2	Sistema di test	58
4.3.3	Dashboard Grafana	58
4.3.4	Relazione tra i sistemi	58
4.4	Criteri di scelta delle tecnologie	59
4.4.1	Sistema principale	59
4.4.2	Sistema di test	59
4.5	Tecnologie	59
4.5.1	Sistema principale	61
4.5.1.1	Backend	61
4.5.1.2	Database	63
4.5.1.3	Deployment	63
4.5.1.4	Strumenti di supporto	64

4.5.2 Sistema di test	64
4.5.2.1 Backend	64
4.5.2.2 Database	65
4.5.2.3 Deployment	65
4.5.2.4 Strumenti di supporto	65
5 Analisi iniziale	67
6 Descrizione del Lavoro Svolto	69
6.1 Struttura del database	69
6.1.1 Tabelle	69
6.1.2 Staging	70
6.1.2.1 promozione	70
6.1.3 Partitioning	71
6.1.4 Indici	71
6.2 Struttura generale del sistema	73
6.2.1 Retriever	73
6.2.1.1 Struttura della pipeline di ingestion	73
6.2.2 Progettazione della ricerca	75
6.2.2.1 Progettazione della ricerca semantica su singola entità	78
6.3 Progettazione della ricerca full text	85
6.3.1 ricerca ibrida	87
6.4 Progettazione della ricerca linked	87
6.5 retriever trial	88
6.5.1 esecuzione query	88
6.5.2 logging risultati	88
6.5.3 Dashboard	88
7 Conclusioni	89
7.1 Consuntivo finale	89
7.2 Raggiungimento degli obiettivi	89
7.3 Requisiti soddisfatti	89
7.4 Rischi occorsi e mitigati	90
7.5 Valutazione personale	90
Glossario	91
Bibliografia	93

Elenco delle Figure

Figura 1	Logo Pat SRL	1
Figura 2	Diagramma del caso d'uso: Ingestion di documenti	18
Figura 3	Diagramma del caso d'uso: Ingestion liste entità	19
Figura 4	Diagramma del caso d'uso: Ingestion lista entità	20
Figura 5	Diagramma del caso d'uso: Caricamento blocco entità ...	21
Figura 6	Diagramma del caso d'uso: Termina ingestion	23
Figura 7	Diagramma del caso d'uso: Ricerca su singola entità	24
Figura 8	Diagramma del caso d'uso: Inserimento query su singola entità	25
Figura 9	Diagramma del caso d'uso: Ricerca semantica	26
Figura 10	Diagramma del caso d'uso: Ricerca ibrida	27
Figura 11	Diagramma del caso d'uso: Ricerca ibrida con modello di re ranking	29
Figura 12	Diagramma del caso d'uso: Ricerca linked	30
Figura 13	Diagramma del caso d'uso: Inserimento query linked ...	31
Figura 14	Diagramma del caso d'uso: Ricerca linked semantica ...	32
Figura 15	Diagramma del caso d'uso: Ricerca linked ibrida	33
Figura 16	Diagramma del caso d'uso: Ricerca linked ibrida con modello di re ranking	34
Figura 17	Diagramma del caso d'uso: Avvia test	36
Figura 18	Diagramma del caso d'uso: Visualizza dashboard performance	36
Figura 19	Diagramma del caso d'uso: Visualizza metriche di performance	38
Figura 20	Diagramma del caso d'uso: Visualizzazione retrieval latency	39
Figura 21	Logo Python	60
Figura 22	Logo Postgres	60
Figura 23	Logo Grafana	61
Figura 24	Logo Fastapi	61

Elenco delle Tabelle

Tabella 1	Requisiti funzionali	44
Tabella 2	Requisiti di qualità	51
Tabella 3	Requisiti di vincolo	51
Tabella 4	Riepilogo dei requisiti.	54
Tabella 5	Consuntivo orario finale.	89
Tabella 6	Rischi occorsi con la loro mitigazione.	90

Elenco dei Codici Sorgente

Codice 1	Codice d'esempio.	7
Codice 2		71
Codice 3		72
Codice 4		72
Codice 5		75
Codice 6		75
Codice 7		76
Codice 8		76
Codice 9		76
Codice 10		77
Codice 11		77
Codice 12		77
Codice 13		78
Codice 14		79
Codice 15		80
Codice 16		81
Codice 17		82
Codice 18		82
Codice 19		83
Codice 20		84
Codice 21		84
Codice 22		85
Codice 23		86
Codice 24		87

Capitolo 1

Introduzione

In questo capitolo descrivo l'azienda, introduco il progetto, e spiego le motivazioni che mi hanno portato a sceglierlo.

1.1 L'azienda

Pat SRL è un'azienda parte del Gruppo Zucchetti, vanta un'esperienza ultra trentennale nello sviluppo di soluzioni software destinate sia ad aziende private che a istituzioni pubbliche. L'azienda si posiziona come partner tecnologico specializzato nella progettazione di piattaforme per la gestione e l'automazione dei processi aziendali e dei servizi di assistenza e supporto.



Figura 1: Logo Pat SRL

1.2 Il progetto

Il progetto ha come obiettivo principale la riprogettazione e la sostituzione dell'attuale architettura dati utilizzata per la persistenza e il recupero delle informazioni all'interno dei prodotti aziendali.

Attualmente, l'impresa adotta una soluzione basata sul paradigma della persistenza poliglotta.

1 Introduzione

Tale architettura prevede l'utilizzo congiunto di due sistemi separati: Postgres per la gestione dei dati strettamente relazionali ed Elasticsearch per l'indicizzazione dei documenti, la ricerca full-text, la gestione degli embedding vettoriali e le operazioni di filtraggio avanzato.

Sebbene questo approccio ibrido sia funzionale e ampiamente utilizzato, la divisione dei carichi di lavoro su motori di database differenti comporta diverse limitazioni architettoniche e operative:

- Denormalizzazione dei dati: la natura orientata ai documenti e non relazionale di Elasticsearch obbliga a replicare e denormalizzare i dati relazionali per consentire un filtraggio efficiente, aumentando la ridondanza e forzando a implementare in modo non ottimale funzioni come i join.
- Overhead di sincronizzazione: il mantenimento della coerenza tra il database primario Postgres e il motore di ricerca Elasticsearch richiede complesse pipeline di allineamento, esponendo il sistema a ritardi di sincronizzazione o disallineamenti.
- Mancanza di garanzie ACID globali: operando su sistemi separati, risulta complesso garantire l'atomicità e la consistenza transazionale durante l'inserimento o l'aggiornamento simultaneo di dati strutturati e vettoriali.

Per superare queste criticità, il progetto esplora la transizione verso un paradigma a database unificato. L'obiettivo è accentrare l'intero carico di lavoro su Postgres sfruttando pgvector, un'estensione open-source che introduce il supporto nativo alla persistenza dei vettori di embedding e alle operazioni di algebra lineare direttamente all'interno dell'ecosistema relazionale.

Nello specifico, il nuovo sistema dovrà soddisfare i seguenti requisiti implementativi:

- Ingestion: sviluppo di un modulo dedicato all'inserimento simultaneo di dati relazionali e documenti.
- Ottimizzazione degli indici: sfruttamento delle capacità di indicizzazione full-text native di Postgres e creazione di indici vettoriali dedicati tramite pgvector per garantire l'efficienza scalabile della ricerca semantica.
- Integrazione relazionale: utilizzo di costrutti SQL per correlare dinamicamente i documenti e i vettori tra le varie entità del sistema.
- Ricerca Ibrida: implementazione di una logica di recupero che combini la precisione lessicale della ricerca testuale con la profondità concettuale della ricerca semantica, fondendo i risultati tramite l'algoritmo di Reciprocal Rank Fusion.
- Ricerca Linked: modalità di ricerca che permette di eseguire in modo automatico una ricerca che analizza ogni entità del sistema e ricostruisce un quadro complessivo dell'informazione tramite join.

1 Introduzione

Al fine di validare rigorosamente l'efficacia di questa nuova architettura unificata, il sistema verrà sottoposto a una fase di benchmarking contro la soluzione attualmente in uso.

I test comparativi si concentreranno sulle seguenti metriche chiave:

- Latenza di interrogazione: misurazione dei tempi di risposta durante la ricerca testuale, semantica e ibrida.
- Velocità di indicizzazione: tempi necessari per l'elaborazione e l'inserimento a database di nuovi record complessi.
- Impatto sullo storage: analisi dello spazio su disco occupato dai dati e dai relativi indici.
- Complessità della pipeline: valutazione qualitativa della semplificazione architetturale.

1.3 Scelta del progetto

Ho scelto questo progetto per 3 ragioni principali:

1. Rilevanza dell'argomento: alla base dei moderni sistemi di Intelligenza Artificiale, come la RAG, che lo utilizzano per fornire contesto ai Large Language Models.
2. Evoluzione di un sistema reale: permette di partecipare all'evoluzione di un software in produzione, che durante il percorso universitario non ho affrontato.
3. Stack tecnologico: offre l'opportunità di operare sul campo con strumenti e framework moderni.

1 Introduzione

Capitolo 2

Descrizione stage

In questo capitolo approfondisco l'organizzazione dello stage, il rapporto con l'azienda e svolgo l'analisi dei rischi.

2.1 Competenze da apprendere

Il tirocinio è strutturato per favorire lo sviluppo di un ventaglio di competenze trasversali, che spaziano dalla progettazione architettuale di alto livello fino all'implementazione tecnica.

Dal punto di vista metodologico, lo stage mira a consolidare le capacità di analisi dei requisiti, astrazione dei problemi complessi e progettazione di soluzioni algoritmiche generali e scalabili, promuovendo inoltre la collaborazione con i tutor.

Sotto il profilo tecnico, il percorso formativo permetterà di acquisire e approfondire le seguenti tematiche:

- Database avanzati: Padronanza nell'utilizzo di Postgres non solo come database relazionale, ma come motore di Information Retrieval vettoriale tramite l'estensione pgvector.
- Progettazione della ricerca ibrida: Studio e valutazione delle tecnologie di indicizzazione. Per garantire un confronto equo e rigoroso con Elasticsearch, oltre alla ricerca full-text nativa di Postgres, verrà esplorata l'integrazione di ParadeDB, una soluzione basata su Postgres che promette capacità di ricerca lessicale altrettanto evolute.
- Testing delle performance: Acquisizione di metodologie per la conduzione di benchmark, definendo metriche di valutazione per misurare le performance e l'efficienza dei sistemi sviluppati.

2.2 Vincoli

Il progetto è soggetto a specifici vincoli architettureali volti a garantire la coerenza con gli obiettivi della ricerca.

I vincoli principali sono i seguenti:

- Utilizzo di pgvector per la ricerca vettoriale, obiettivo principale del progetto;

2 Descrizione stage

- Utilizzo della ricerca full-text nativa di Postgres, per semplicità di licensing.

2.3 Pianificazione

Lo stage si articola in 320 ore distribuite su otto settimane da 40 ore.

La pianificazione, derivata dal piano di lavoro, è la seguente:

1. Prima Settimana - Studio e analisi iniziale del nuovo e del vecchio stack tecnologico e setup (40 ore): studio delle tecnologie, setup dell'ambiente di lavoro locale e prove generiche;
2. Seconda Settimana - Analisi comparativa dettagliata di elastic search e Postgres con tentativo di modellazione di un sottoinsieme limitato del problema;
3. Terza Settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale;
4. Quarta settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale e ottimizzazione tramite indici;
5. Quinta settimana - Progettazione di alto livello del sistema e inizio della codifica del modulo di ingestion;
6. Sesta settimana - Completamento della codifica del modulo di ingestion e dei moduli di ricerca, completa delle relative interfacce;
7. Settima settimana - Progettazione e implementazione del sistema di test partendo da dei dati locali fittizi;
8. Ottava settimana - Benchmarking, realizzazione della dashboard e deployment.

2.4 Analisi dei rischi

I rischi identificati per questo progetto sono classificati con un codice progressivo della forma **RN**, dove **N** è un numero intero incrementale che parte da 01, e decorati con una probabilità di occorrenza, un impatto e una strategia di mitigazione.

Ogni rischio è stato analizzato tenendo conto della complessità di comprendere i casi d'uso del sistema di paragone Elastic, delle sue scelte implementative dovute allo stack tecnologico e dalla comprensione degli interessi sperimentativi dell'impresa.

Codice : R01

Incompletezza o ambiguità dei requisiti

- **Descrizione:**

I requisiti descritti nel piano di lavoro possono risultare incompleti o ambigui, portando a implementazioni non coerenti con le aspettative aziendali.

- **Mitigazione:**

I requisiti vengono analizzati e chiariti prima delle attività di codifica. Viene data priorità ad attività di studio teorico dello stack tecnologico e alla realizzazione di prototipi a diversi livelli di complessità. Sono previsti incontri iterativi con il tutor per definire chiaramente i confini del progetto e le interfacce di comunicazione.

- **Probabilità:** Medio-Alta

- **Impatto:** Medio

Codice : R02

Sovradimensionamento del progetto rispetto alle tempistiche

- **Descrizione:**

Le attività previste potrebbero rivelarsi eccessive rispetto al monte ore disponibile e alle tempistiche di apprendimento, con il rischio di non completare tutti gli obiettivi prefissati.

- **Mitigazione:**

Le attività sono suddivise per priorità (requisiti obbligatori, desiderabili, opzionali). In caso di ritardi o imprevisti strutturali, solo le attività a priorità inferiore e non vincolanti per lo scopo principale del progetto subiranno ridimensionamenti.

- **Probabilità:** Media

- **Impatto:** Alto

2 Descrizione stage

Codice : R03

Difficoltà nel coordinamento interno

- **Descrizione:**

La comunicazione con il tutor aziendale o con gli stakeholder potrebbe risultare discontinua, rallentando il processo decisionale tecnico e causando incomprensioni.

- **Mitigazione:**

Vengono pianificati incontri periodici di allineamento, accompagnati da aggiornamenti asincroni frequenti sullo stato di avanzamento. Eventuali blocchi tecnici vengono segnalati tempestivamente senza attendere la riunione successiva.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R04

Curva di apprendimento delle tecnologie

- **Descrizione:**

L'assimilazione delle tecnologie necessarie (es. pgvector, fts Postgres) potrebbe richiedere un tempo di studio superiore alle stime iniziali.

- **Mitigazione:**

Le prime settimane dello stage includono ore dedicate esplicitamente allo studio della documentazione ufficiale, alla schematizzazione delle architetture e alla realizzazione di proof of concept isolati.

- **Probabilità:** Bassa

- **Impatto:** Medio

Codice : R05

Incompatibilità o difficoltà di integrazione con il sistema legacy

- **Descrizione:**

Il modulo realizzato dovrà essere testato in un ambiente paragonabile a quello di produzione; potrebbero emergere attriti o incompatibilità nell'interfacciamento con il sistema esistente.

- **Mitigazione:**

Confronto continuo con il team di sviluppo per comprendere a fondo le funzionalità da implementare. Adozione del design pattern «Adapter» fin dalle prime fasi di progettazione per disaccoppiare la forma dei dati accettata dal sistema con la forma dei dati che deve essere accettata dall'esterno.

- **Probabilità:** Media

- **Impatto:** Medio

Codice : R06

Saturazione delle risorse durante i benchmark

- **Descrizione:**

L'esecuzione dei test comparativi su volumi di dati enterprise (fino a 500.000 record) potrebbe causare la saturazione delle risorse hardware dell'ambiente di test, falsando le metriche o causando crash di sistema.

- **Mitigazione:**

I test verranno eseguiti in modo incrementale su dataset di dimensioni crescenti.

Verrà implementato un monitoraggio attivo delle risorse tramite il framework di observability per individuare eventuali memory leak o configurazioni sub-ottimali degli indici vettoriali prima dei test massivi.

Inoltre è previsto il deployment del sistema di information retrieval su server remoto.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R07

Rappresentatività dei dati di test e accesso limitato alla produzione

- **Descrizione:**

L'utilizzo di dati generati appositamente per le fasi di test locali, reso necessario dai vincoli di privacy aziendale, potrebbe non riflettere a pieno la reale complessità e varianza del dominio. Questa discrepanza rischia di alterare la valutazione dell'accuratezza degli embedding e di non far emergere casistiche limite verosimili, portando a possibili divergenze di performance tra l'ambiente di sviluppo e le aspettative finali.

- **Mitigazione:**

La validazione seguirà un approccio a due fasi: l'architettura dovrà innanzitutto superare i benchmark di riferimento in ambiente locale utilizzando dei dati di test.

A valle di questo consolidamento, le misurazioni definitive verranno eseguite sui dati reali operando esclusivamente all'interno dei server proprietari aziendali, nel pieno rispetto delle direttive di sicurezza.

- **Probabilità:** Alta

- **Impatto:** Medio

Codice : R08

Difficoltà nella valutazione oggettiva dei risultati di Retrieval

- **Descrizione:**

L'assenza di metriche standardizzate o un'errata interpretazione dei risultati potrebbe portare a conclusioni soggettive, rendendo impossibile valutare se il nuovo sistema eguaglia o supera la soluzione precedente.

- **Mitigazione:**

Le metriche di valutazione quantitativa verranno definite esplicitamente nella fase di analisi dei requisiti.

Queste corrispondono alle metriche attualmente usate per la valutazione del sistema attuale

È inoltre previsto un caso d'uso specifico per la produzione di una dashboard di monitoraggio, con criteri di accettazione e soglie minime di qualità chiaramente prestabiliti.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R09

Sbilanciamento metodologico nel confronto

- **Descrizione:**

Essendo Postgres nato come RDBMS ed Elasticsearch come motore di ricerca con analizzatori linguistici avanzati, testare scenari non calibrati rischia di favorire asimmetricamente una delle due tecnologie, invalidando l'equità scientifica del benchmark.

- **Mitigazione:**

Il set di query e il benchmark verranno definiti e «congelati» a priori, in stretta parità di funzionalità con l'attuale utilizzo in produzione, evitando scenari costruiti ad hoc per avvantaggiare una specifica piattaforma, cercheranno solo di rappresentare lo scenario reale.

- **Probabilità:** Alta

- **Impatto:** Alto

Codice : R10

Bias tecnologico e deriva dei requisiti

- **Descrizione:**

Esiste il rischio di adattare inconsciamente l'implementazione per favorire le peculiarità di Postgres, introducendo logiche non necessarie o deviazioni dai requisiti originali solo per giustificare l'adozione della nuova tecnologia.

- **Mitigazione:**

Adozione dell'architettura esagonale. Isolando la logica di dominio dall'infrastruttura di persistenza, si garantisce che i dettagli implementativi di Postgres non inquinino il comportamento atteso del sistema.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R11

Dispersione del perimetro di test su tecnologie alternative

- **Descrizione:**

L'evoluzione rapida del panorama RAG solleva l'interrogativo su alternative tecnologiche, come Open-Search.

La loro valutazione va fuori dagli interessi dell'impresa per questo specifico tirocinio e rischia di aggiungere attività di studio non utili alla realizzazione del progetto.

- **Mitigazione:**

I confini del tirocinio sono rigidamente circoscritti al confronto diretto tra la soluzione in uso, basata su Elasticsearch, e le tecnologie che l'impresa vuole valutare, Postgres + pgvector.

Eventuali tecnologie alternative verranno affrontate esclusivamente a livello teorico.

- **Probabilità:** Bassa

- **Impatto:** Medio

Codice : R12

Sovrapposizione tra le performance di Retrieval e Generation

- **Descrizione:**

L'architettura RAG si compone di due fasi: la fase di retrieval che si occupa del recupero di informazioni e la generazione della risposta. Nel sistema attuale solo la parte di retrieval e ingestion di documenti è collegata strettamente a Elasticsearch, le altre parti del sistema sono gestite in Python puro.

- **Mitigazione:**

Vengono posti dei rigidi confini sul sistema da implementare. L'implementazione, l'analisi e il testing si concentreranno unicamente sulla componente di Retriever e sul relativo modulo di ingestion. La valutazione ignorerà la qualità dell'output generativo dell'LLM, misurando esclusivamente la pertinenza dei documenti e la velocità di recupero.

- **Probabilità:** Bassa

- **Impatto:** Alto

2 Descrizione stage

Capitolo 3

Analisi dei requisiti

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

3.1 Analisi degli Utenti

Al fine di modellare correttamente le interazioni e definire i confini del sistema oggetto del tirocinio, è necessario individuare gli attori coinvolti. In accordo con le metodologie dell'ingegneria del software, gli attori comprendono sia gli utenti umani sia i sistemi software esterni che interagiscono direttamente con l'architettura.

Gli attori sono classificati in due categorie:

- Attori primari: entità che avviano i casi d'uso e richiedono un servizio al sistema.
- Attori secondari: servizi o sistemi esterni invocati dal sistema per completare una specifica elaborazione.

3.1.1 Attori primari

- **Companion**

Con il termine **Companion** si identifica l'applicativo aziendale di Intelligenza Artificiale di livello superiore. Questo attore software è il client principale: è il software applicativo reale che utilizza il modulo di information retrieval.

Companion interagisce con il sistema tramite chiamate api.

Companion interagisce con il sistema per due scopi fondamentali: delegare l'ingestione dei dati documentali e interrogare la base di conoscenza tramite la pipeline RAG per ottenere il contesto necessario alla generazione delle risposte.

3 Analisi dei requisiti

- **Supervisore**

Rappresenta l'utente tecnico qualificato. Il suo ruolo è quello di interfacciarsi con il sistema a scopo di analisi, validazione e benchmarking. Il Supervisore avvia i test prestazionali, monitora l'infrastruttura e raccoglie le metriche necessarie per valutare e comparare le prestazioni del nuovo database unificato rispetto alla soluzione correntemente usata in produzione.

3.1.2 Attori secondari

- **Modello di embedding**

Si tratta di un attore software esterno che fornisce il servizio di vettorizzazione. Il sistema oggetto dello stage invoca questo attore delegandogli il compito computazionale di trasformare i chunk di testo grezzo in vettori numerici, questi vettori verranno poi usati per popolare l'indice vettoriale e per l'esecuzione della ricerca semantica.

- **Modello di re-ranking**

Si tratta di un attore software esterno che realizza la funzione di merge dei risultati della ricerca semantica e della ricerca full-text

3.2 Casi d'uso

In questa sezione vengono definiti i casi d'uso del sistema. Ogni caso d'uso è univocamente identificato da una sigla progressiva (es. **UC-X**) ed è corredato da una scheda descrittiva analitica. Gli attori menzionati fanno diretto riferimento alle definizioni riportate nella sezione Capitolo 3.1.

3.2.1 Struttura della scheda descrittiva

Ciascun caso d'uso è documentato attraverso le informazioni elencate nella tabella seguente. Al fine di mantenere la documentazione concisa, i campi non rilevanti o non applicabili a uno specifico scenario verranno omessi.

Campo	Descrizione
Grafico UML	Rappresentazione visiva dello scenario tramite diagramma UML dei casi d'uso.
Attore principale	Entità esterna che avvia l'interazione con il sistema per raggiungere un obiettivo specifico.

3 Analisi dei requisiti

Campo	Descrizione
Attore secondario	Sistema o servizio esterno invocato dal sistema per completare una determinata funzionalità.
Scenario principale	La sequenza nominale di operazioni necessaria per portare a compimento il caso d'uso senza interruzioni.
Precondizioni	Stato del sistema o requisiti che devono essere necessariamente soddisfatti prima dell'avvio del caso d'uso.
Postcondizioni	Stato del sistema o modifiche persistenti che si verificano al termine della corretta esecuzione dello scenario principale.
Scenario alternativo	Flussi di esecuzione secondari che deviano dallo scenario base, tipicamente per gestire anomalie, errori o percorsi non standard.
Inclusioni	Relazione verso un altro caso d'uso, il cui comportamento è obbligatoriamente e incondizionatamente incorporato nello scenario corrente.
Estensioni	Relazione che introduce un comportamento opzionale o alternativo, attivato unicamente al verificarsi di specifiche condizioni.
Specializzazioni	Varianti strutturali del caso d'uso generale. I casi d'uso specializzati sono tra loro mutualmente esclusivi ed ereditano i comportamenti dello scenario base.
Trigger	L'evento, l'azione o la condizione scatenante che innesca l'esecuzione del caso d'uso.

3 Analisi dei requisiti

3.2.2 Lista dei casi d'Uso

UC01: Ingestion di documenti

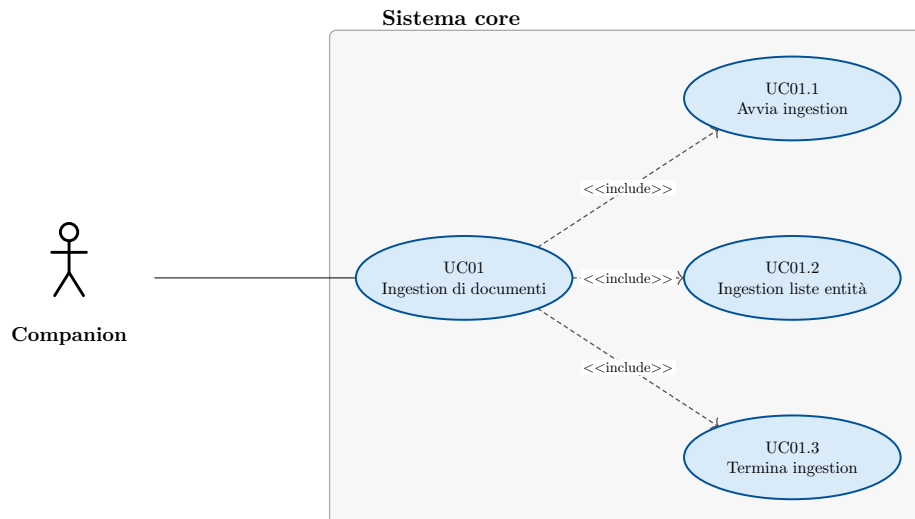


Figura 2: Diagramma del caso d'uso: Ingestion di documenti

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
 - Nel sistema non ci sono sessioni di ingestion di documenti attive
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni ricevute
 - Il sistema ha salvato gli embedding relativi ai dati
 - Il sistema ha salvato le informazioni di lingua relative ai dati
 - Il sistema ha reso le informazioni disponibili per la ricerca
- **Scenario principale:**
 1. Companion avvia il processo di ingestion dei dati → UC01.1
 2. Companion carica le liste di entità → UC01.2
 3. Companion termina il processo di ingestion → UC01.3
 4. Companion viene notificato del corretto salvataggio dei dati.
- **Inclusioni:**
 - UC01.1
 - UC01.2
 - UC01.3
- **Trigger:** Companion vuole caricare dei documenti sul sistema

UC01.1: Avvia ingestion

- **Attore principale:** Companion

3 Analisi dei requisiti

- **Precondizioni:**
 - Il sistema è attivo
 - Nel sistema non ci sono sessioni di ingestion di documenti attive
- **Postcondizioni:**
 - Nel sistema è attiva una sessione di ingestion di documenti
- **Scenario principale:**
 1. Companion chiede di avviare una sessione di ingestion
 2. Il sistema avvia la sessione di ingestion
 3. Companion viene notificato della corretta apertura del sistema

UC01.2: Ingestion liste entità

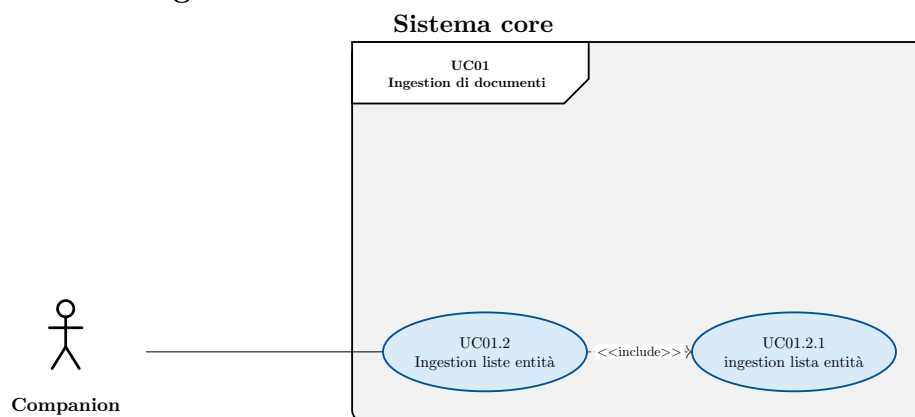


Figura 3: Diagramma del caso d'uso: Ingestion liste entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative alle liste di entità
 - Il sistema ha salvato gli embedding relativi alle liste di entità
 - Il sistema ha salvato le informazioni di lingua relative alle liste di entità
- **Scenario principale:**
 1. Companion carica le liste di entità
 - 1.1. Companion carica una lista di entità → UC01.2.1
- **Inclusioni:**
 - UC01.2.1

3 Analisi dei requisiti

UC01.2.1: Ingestion lista entità

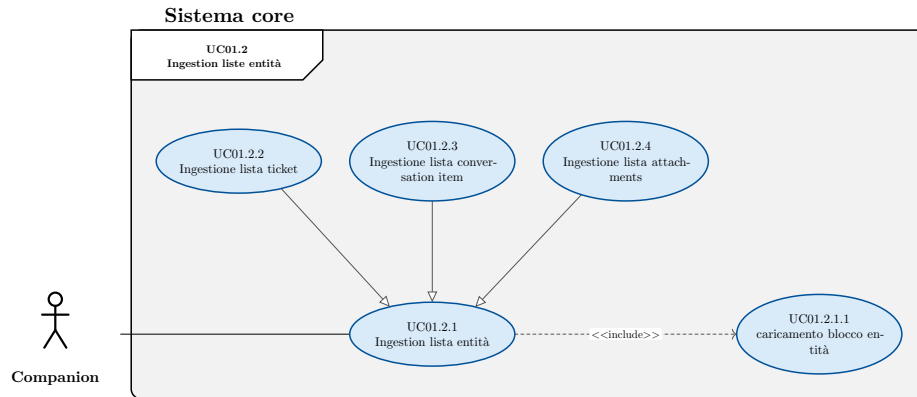


Figura 4: Diagramma del caso d'uso: Ingestion lista entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative alla lista di entità
 - Il sistema ha salvato gli embedding relativi alla lista di entità
 - Il sistema ha salvato le informazioni di lingua relative alla lista di entità
- **Scenario principale:**
 1. Companion carica la lista di entità
 - 1.1. Companion carica un blocco di entità → UC01.2.1.1
- **Inclusioni:**
 - UC01.2.1.1
- **Specializzazioni:**
 - UC01.2.2
 - UC01.2.3
 - UC01.2.4

3 Analisi dei requisiti

UC01.2.1.1: Caricamento blocco entità

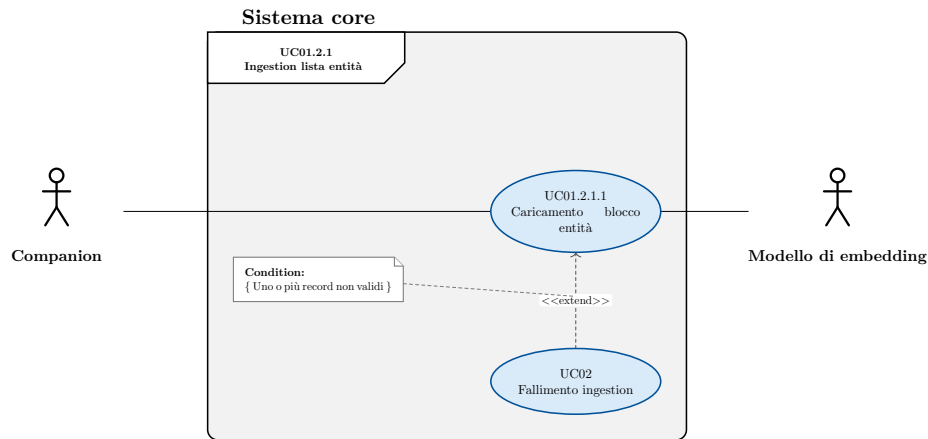


Figura 5: Diagramma del caso d'uso: Caricamento blocco entità

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il processo di caricamento lista di entità è in corso
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative al blocco di entità
 - Il sistema ha salvato gli embedding relativi al blocco di entità
 - Il sistema ha salvato le informazioni di lingua relative al blocco di entità
- **Scenario principale:**
 1. Companion carica un blocco di entità
 2. Il sistema salva i dati dei entità
 3. Il sistema calcola l'embedding da associare ai chunk di testo usando il modello di embedding
 4. Il sistema rileva le informazioni di lingua da applicare ai chunk
 - 4.1. Il sistema può usare l'informazione linguistica presente nei dati caricati
 - 4.2. Il sistema può usare rileva la lingua del testo
 5. Il sistema associa gli embedding al relativo frammento di testo
 6. Il sistema associa la lingua al relativo frammento di testo
 7. Il sistema salva il blocco di entità
- **Scenari alternativi:**
 - Uno o più record del blocco non sono validi → UC02
- **Estensioni:**
 - UC02

UC01.2.2: Ingestione lista ticket

3 Analisi dei requisiti

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative alla lista di ticket
 - Il sistema ha salvato gli embedding relativi alla lista di ticket
 - Il sistema ha salvato le informazioni di lingua relative alla lista di ticket
- **Scenario principale:**
 1. Companion carica la lista dei ticket

UC01.2.3: Ingestione lista conversation item

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative alla lista di conversation item
 - Il sistema ha salvato gli embedding relativi alla lista di conversation item
 - Il sistema ha salvato le informazioni di lingua relative alla lista di conversation item
- **Scenario principale:**
 1. Companion carica la lista dei conversation item
- **Inclusioni:**

UC01.2.4: Ingestione lista attachments

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema ha salvato le informazioni relative alla lista di attachment
 - Il sistema ha salvato gli embedding relativi alla lista di attachment
 - Il sistema ha salvato le informazioni di lingua relative alla lista di attachment
- **Scenario principale:**
 1. Companion carica la lista dei attachment
- **Inclusioni:**

3 Analisi dei requisiti

UC01.3: Termina ingestion

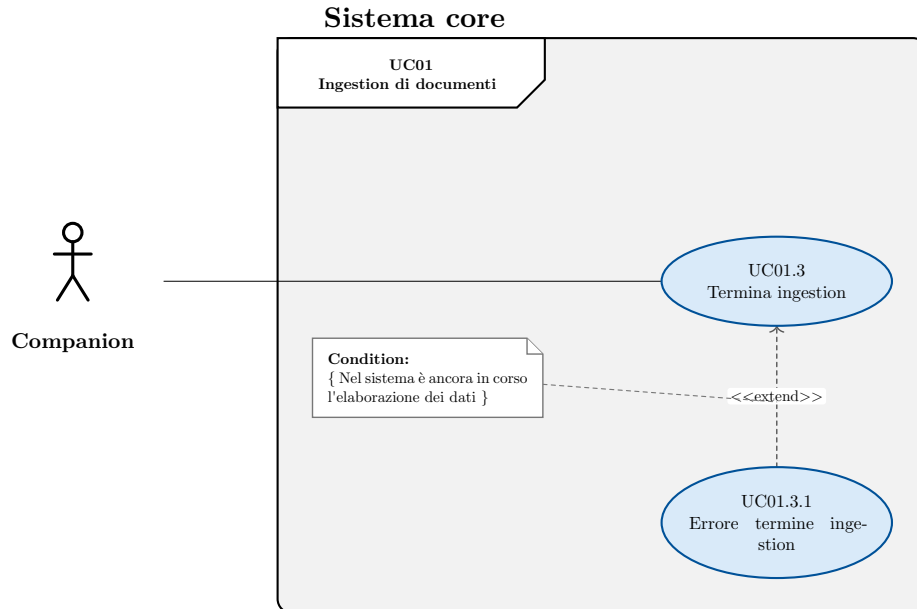


Figura 6: Diagramma del caso d'uso: Termina ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
 - Nel sistema i dati inseriti sono stati resi disponibili per la ricerca
- **Postcondizioni:**
 - Nel sistema viene chiusa la sessione di ingestion
- **Scenario principale:**
 1. Companion chiede il termine della sessione di ingestion
 2. Il sistema rende i dati disponibili per la ricerca
 3. Il sistema chiude la sessione di ingestion
- **Scenari alternativi:**
 - Nel sistema è ancora in corso l'elaborazione di dati → UC01.3.1
- **Estensioni:**
 - UC01.3.1

UC01.3.1: Errore termine ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è attiva una sessione di ingestion
 - Nel sistema i dati inseriti sono stati resi disponibili per la ricerca
- **Postcondizioni:**
 - Nel sistema rimane attiva la sessione di ingestion
 - Companion viene notificato dell'errore

3 Analisi dei requisiti

- **Scenario principale:**

1. Companion chiede il termine della sessione di ingestion
2. Il sistema rileva che è ancora in corso l'elaborazione dei dati
3. Companion riceve una notifica dell'errore

UC02: Fallimento ingestion

- **Attore principale:** Companion

- **Precondizioni:**

- Il processo di caricamento lista di entità è in corso

- **Postcondizioni:**

- Il sistema non ha salvato i record non validi
- Companion riceve un messaggio di errore esplicativo

- **Scenario principale:**

1. Companion carica un blocco di entità
2. Rileva degli errori relativi a uno o più record del blocco
3. Companion riceve un errore esplicativo relativo ai record che hanno generato errori

UC03: Ricerca su singola entità

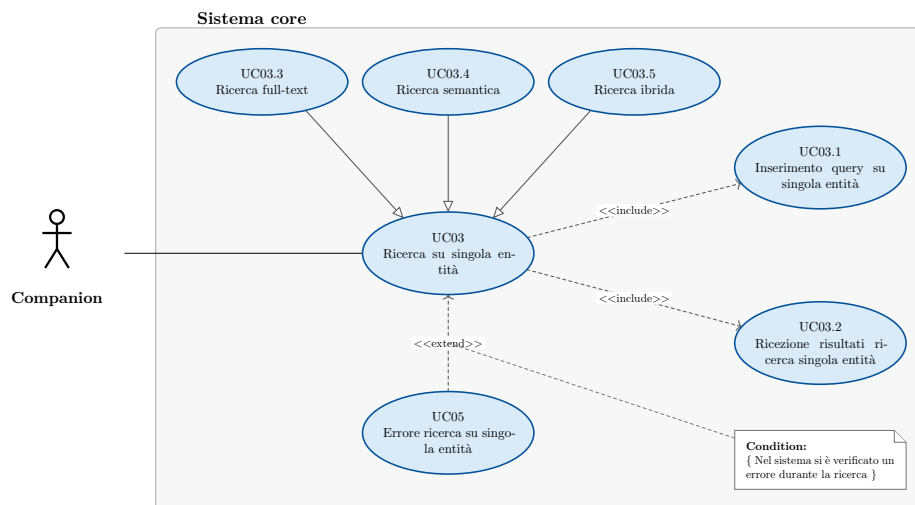


Figura 7: Diagramma del caso d'uso: Ricerca su singola entità

- **Attore principale:** Companion

- **Precondizioni:**

- Il sistema è attivo

- **Postcondizioni:**

- Companion ha ricevuto i risultati della ricerca

3 Analisi dei requisiti

- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Il sistema valida la query
 3. Il sistema esegue la query
 4. Companion riceve i risultati → UC03.2
- **Scenari alternativi:**
 - Errore durante la ricerca → UC05
- **Inclusioni:**
 - UC03.1
 - UC03.2
- **Estensioni:**
 - UC04
- **Specializzazioni:**
 - UC03.3
 - UC03.4
 - UC03.5
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

UC03.1: Inserimento query su singola entità

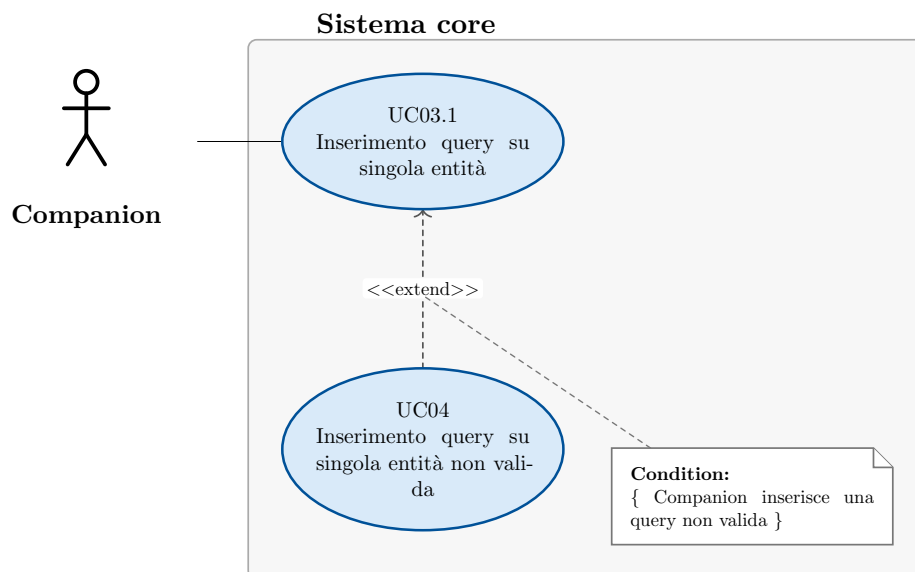


Figura 8: Diagramma del caso d'uso: Inserimento query su singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su una singola entità
- **Postcondizioni:**
 - Il sistema ha elaborato la query
- **Scenario principale:**
 1. Companion inserisce la query di ricerca

3 Analisi dei requisiti

- **Scenari alternativi:**
 - Errore query non valida → UC04
- **Estensioni:**
 - UC04

UC03.2: Ricezione risultati ricerca singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su singola entità
 - Il sistema ha eseguito la query
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Il sistema ritorna la lista dei risultati prodotti dalla ricerca
 2. Companion riceve i risultati della ricerca

UC03.3: Ricerca full text

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Il sistema esegue la query valutando la pertinenza in base alla ricerca full text
 3. Il sistema può ottimizzare la ricerca basandosi sulla lingua
 4. Companion riceve i risultati → UC03.2
- **Trigger:** Companion vuole eseguire una ricerca full-text su una singola entità

UC03.4: Ricerca semantica

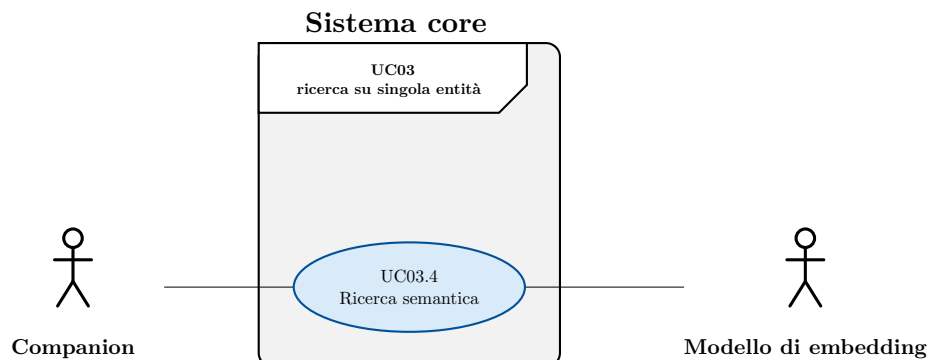


Figura 9: Diagramma del caso d'uso: Ricerca semantica

3 Analisi dei requisiti

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 3. Companion riceve i risultati → UC03.2
- **Trigger:** Companion vuole eseguire una ricerca semantica su una singola entità

UC03.5: Ricerca ibrida

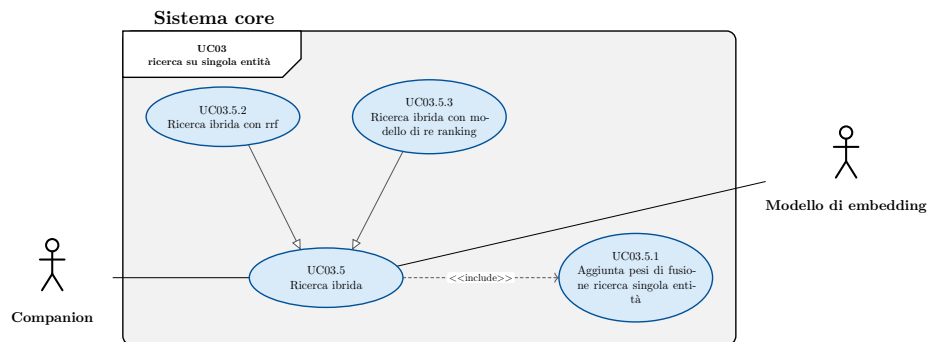


Figura 10: Diagramma del caso d'uso: Ricerca ibrida

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → UC03.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati
 6. Companion riceve i risultati → UC03.2

3 Analisi dei requisiti

- **Inclusioni:**
 - UC03.5.1
- **Specializzazioni:**
 - UC03.5.2
 - UC03.5.3
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

UC03.5.1: Aggiunta pesi di fusione ricerca singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Companion sta eseguendo una ricerca ibrida su singola entità
- **Postcondizioni:**
 - Companion ha inserito i pesi da usare durante la fusione dei risultati di ricerca
- **Scenario principale:**
 - Companion inserisce il peso da applicare alla ricerca full-text
 - Companion inserisce il peso da applicare alla ricerca semantica

UC03.5.2: Ricerca ibrida con rrf

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → UC03.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati tramite RRF
 6. Companion riceve i risultati → UC03.2
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

UC03.5.3: Ricerca ibrida con modello di re ranking

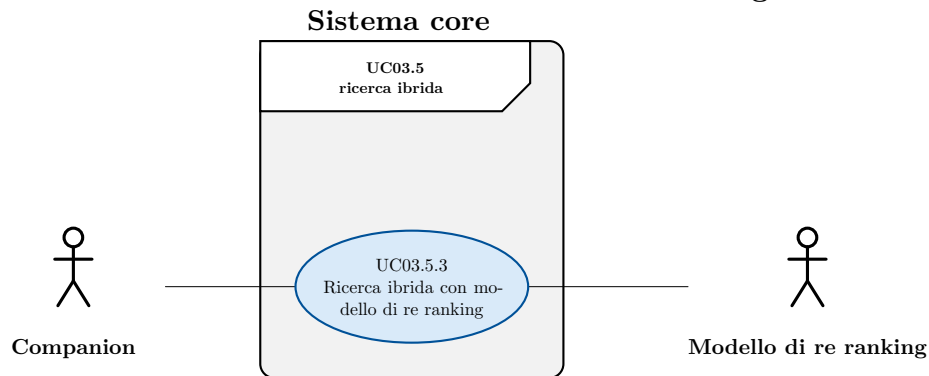


Figura 11: Diagramma del caso d'uso: Ricerca ibrida con modello di re ranking

- **Attore principale:** Companion
- **Attore secondario:** Modello di re-ranking
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → UC03.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati affidandosi a un modello di re-ranking esterno
 6. Companion riceve i risultati → UC03.2
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

UC04: Inserimento query su singola entità non valida

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su una singola entità
 - Companion ha inserito una query non valida
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore

3 Analisi dei requisiti

- **Scenario principale:**
 1. Il sistema rileva la non validità della query
 2. Il sistema interrompe la ricerca
 3. Companion riceve un messaggio d'errore esplicativo

UC05: Errore ricerca su singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su una singola entità
 - Nel sistema si è verificato un errore durante la ricerca
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore
- **Scenario principale:**
 1. Il sistema interrompe la ricerca
 2. Companion riceve un messaggio d'errore esplicativo

UC06: Ricerca linked

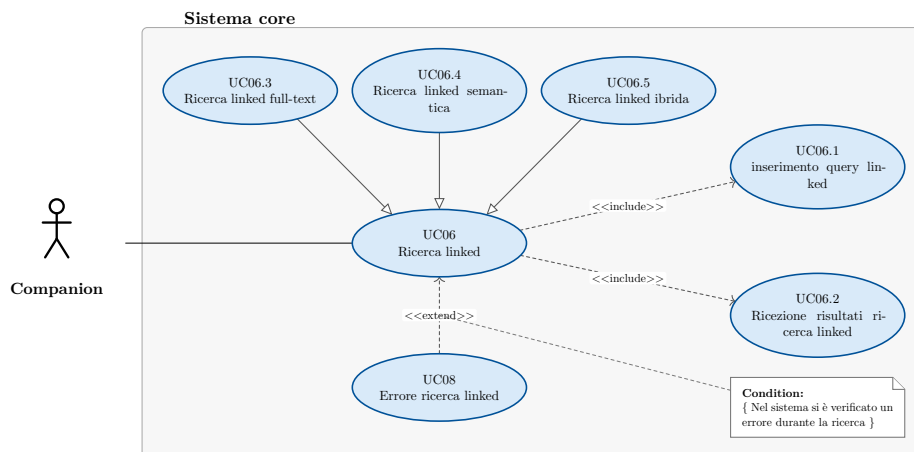


Figura 12: Diagramma del caso d'uso: Ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Il sistema valida la query
 3. Il sistema esegue la query
 4. Companion riceve i risultati → UC06.2
- **Scenari alternativi:**
 - Errore durante la ricerca → UC08

3 Analisi dei requisiti

- **Inclusioni:**
 - UC06.1
 - UC06.2
- **Estensioni:**
 - UC08
- **Specializzazioni:**
 - UC06.3
 - UC06.4
 - UC06.5
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

UC06.1: Inserimento query linked

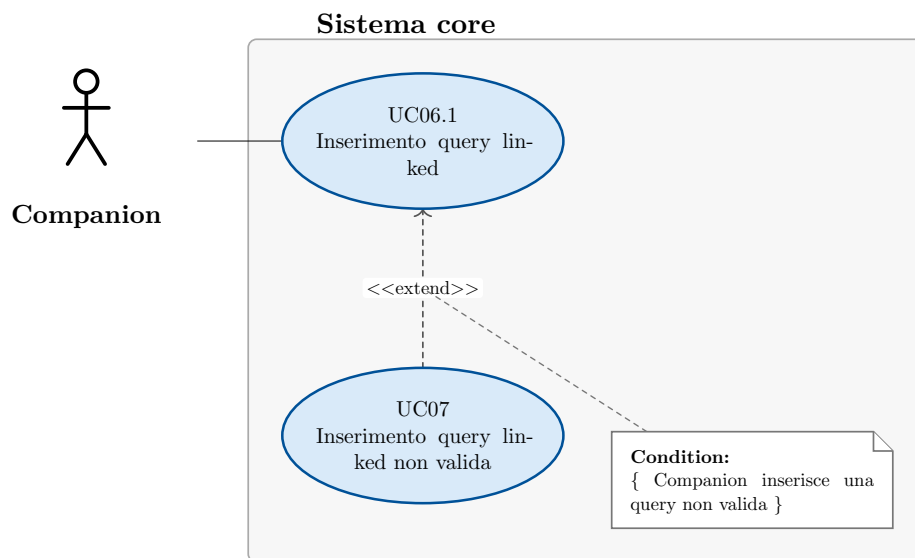


Figura 13: Diagramma del caso d'uso: Inserimento query linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca linked
- **Postcondizioni:**
 - Il sistema ha elaborato la query
- **Scenario principale:**
 1. Companion inserisce la query di ricerca
- **Scenari alternativi:**
 - Errore query non valida → UC07
- **Estensioni:**
 - UC07

UC06.2: Ricezione risultati ricerca linked

- **Attore principale:** Companion

3 Analisi dei requisiti

- **Precondizioni:**
 1. Il sistema ha eseguito la query
- **Postcondizioni:**
 1. Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion ritorna la lista dei risultati prodotti dalla ricerca

UC06.3: Ricerca linked full text

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Il sistema esegue la query valutando la pertinenza in base alla ricerca full text
 3. Il sistema può ottimizzare la ricerca basandosi sulla lingua
 4. Companion riceve i risultati → UC06.2
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

UC06.4: Ricerca linked semantica

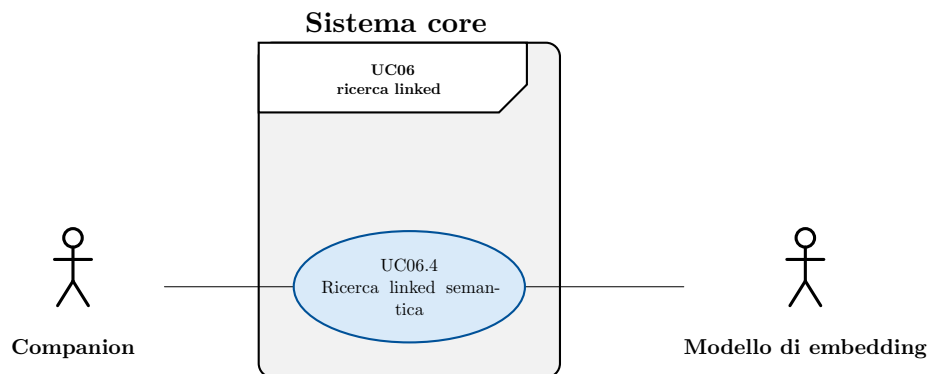


Figura 14: Diagramma del caso d'uso: Ricerca linked semantica

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.

3 Analisi dei requisiti

- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 3. Companion riceve i risultati → UC06.2
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

UC06.5: Ricerca linked ibrida

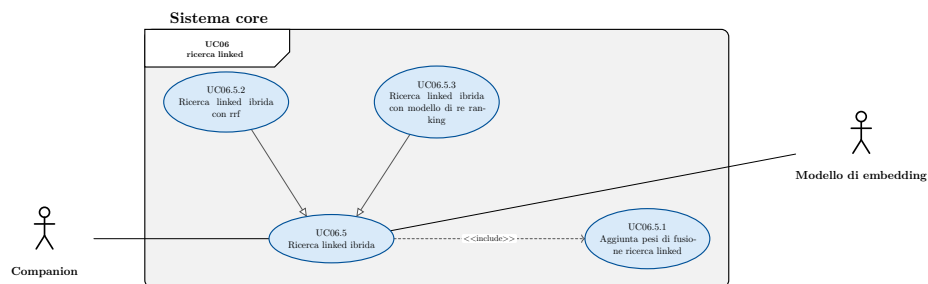


Figura 15: Diagramma del caso d'uso: Ricerca linked ibrida

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca ibrida e full-text → UC06.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati
 6. Companion riceve i risultati → UC06.2
- **Inclusioni:**
 - UC06.5.1
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

UC06.5.1: Aggiunta pesi di fusione ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Companion sta eseguendo una ricerca ibrida in modalità linked

3 Analisi dei requisiti

- **Postcondizioni:**
 - Companion ha inserito i pesi da usare durante la fusione dei risultati di ricerca
- **Scenario principale:**
 - Companion inserisce il peso da applicare alla ricerca full-text
 - Companion inserisce il peso da applicare alla ricerca semantica

UC06.5.2: Ricerca linked ibrida con rrf

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca ibrida e full-text → UC06.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati tramite RRF
 6. Companion riceve i risultati → UC06.2
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

UC06.5.3: Ricerca linked ibrida con modello di re ranking

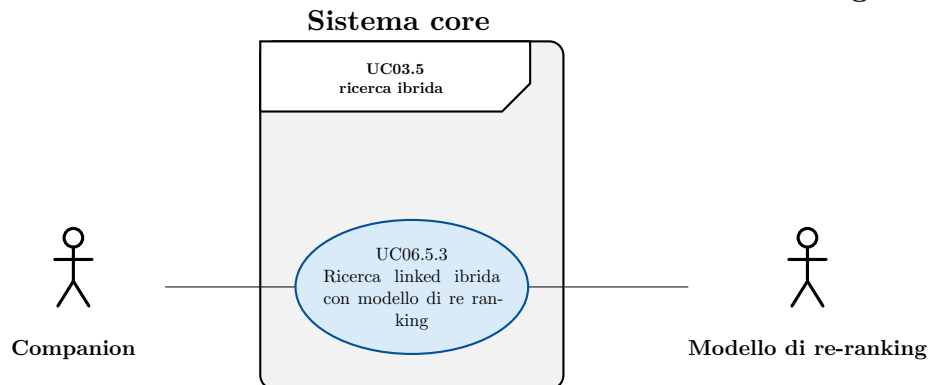


Figura 16: Diagramma del caso d'uso: Ricerca linked ibrida con modello di re ranking

- **Attore principale:** Companion
- **Attore secondario:** Modello di re-ranking
- **Precondizioni:**
 - Il sistema è attivo

3 Analisi dei requisiti

- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca.
- **Scenario principale:**
 1. Companion inserisce una query → UC06.1
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca ibrida e full-text → UC06.5.1
 3. Il sistema esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Il sistema esegue la query valutando la pertinenza in base alla ricerca full-text
 5. Il sistema fonde i risultati tramite un modello di re-ranking
 6. Companion riceve i risultati → UC06.2
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

UC07: Inserimento query linked non valida

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su una singola entità
 - Companion ha inserito una query non valida
- **Postcondizioni:**
 - Companion riceve notifica esplicita dell'errore
- **Scenario principale:**
 1. Il sistema rileva la non validità della query
 2. Il sistema interrompe la ricerca
 3. Companion riceve un messaggio d'errore esplicativo

UC08: Errore ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema è in corso una ricerca su una singola entità
 - Nel sistema si è verificato un errore durante la ricerca
- **Postcondizioni:**
 - Companion riceve notifica esplicita dell'errore
- **Scenario principale:**
 1. Il sistema interrompe la ricerca
 2. Companion riceve un messaggio d'errore esplicativo

3 Analisi dei requisiti

UC09: Avvia test

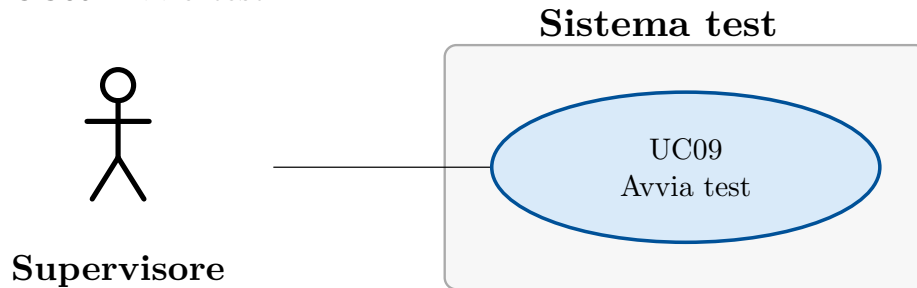


Figura 17: Diagramma del caso d'uso: Avvia test

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
 - Il sistema core è attivo
 - Nel sistema non ci sono altre run di test attive
- **Postcondizioni:**
 - Il sistema ha avviato l'esecuzione delle ricerche di test
- **Scenario principale:**
 1. Il supervisore seleziona l'avvio dei test
 2. Il sistema avvia i test
- **Trigger:** Il supervisore vuole avviare un test di performance

UC10: Visualizza dashboard performance

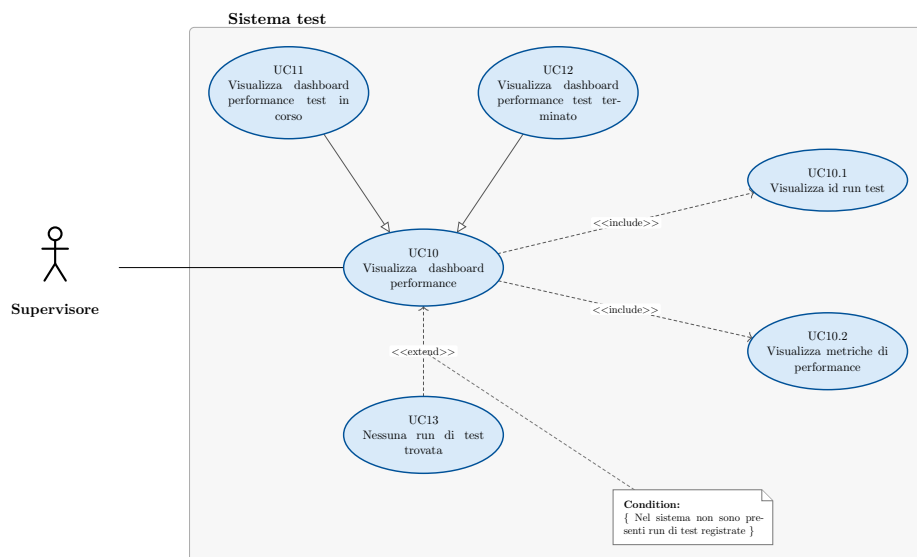


Figura 18: Diagramma del caso d'uso: Visualizza dashboard performance

- **Attore principale:** Supervisore

3 Analisi dei requisiti

- **Precondizioni:**
 - Il sistema di test è attivo
 - Il sistema core è attivo
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato dell'ultima run di test avviata
- **Scenario principale:**
 1. Il supervisore visualizza l'id della run di test → UC10.1
 2. Il supervisore visualizza la lista delle metriche → UC10.2
- **Scenari alternativi:**
 - Nel sistema non vi sono run di test registrate → UC13
- **Inclusioni:**
 - UC10.1
 - UC10.2
- **Estensioni:**
 - UC13
- **Specializzazioni:**
 - UC11
 - UC12

UC10.1: Visualizza id run test

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la dashboard delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato l'id relativo alla run di test a cui si riferiscono le performance
- **Scenario principale:**
 - Il supervisore visualizza l'id relativo alla run di test a cui si riferiscono le performance

3 Analisi dei requisiti

UC10.2: Visualizza metriche di performance

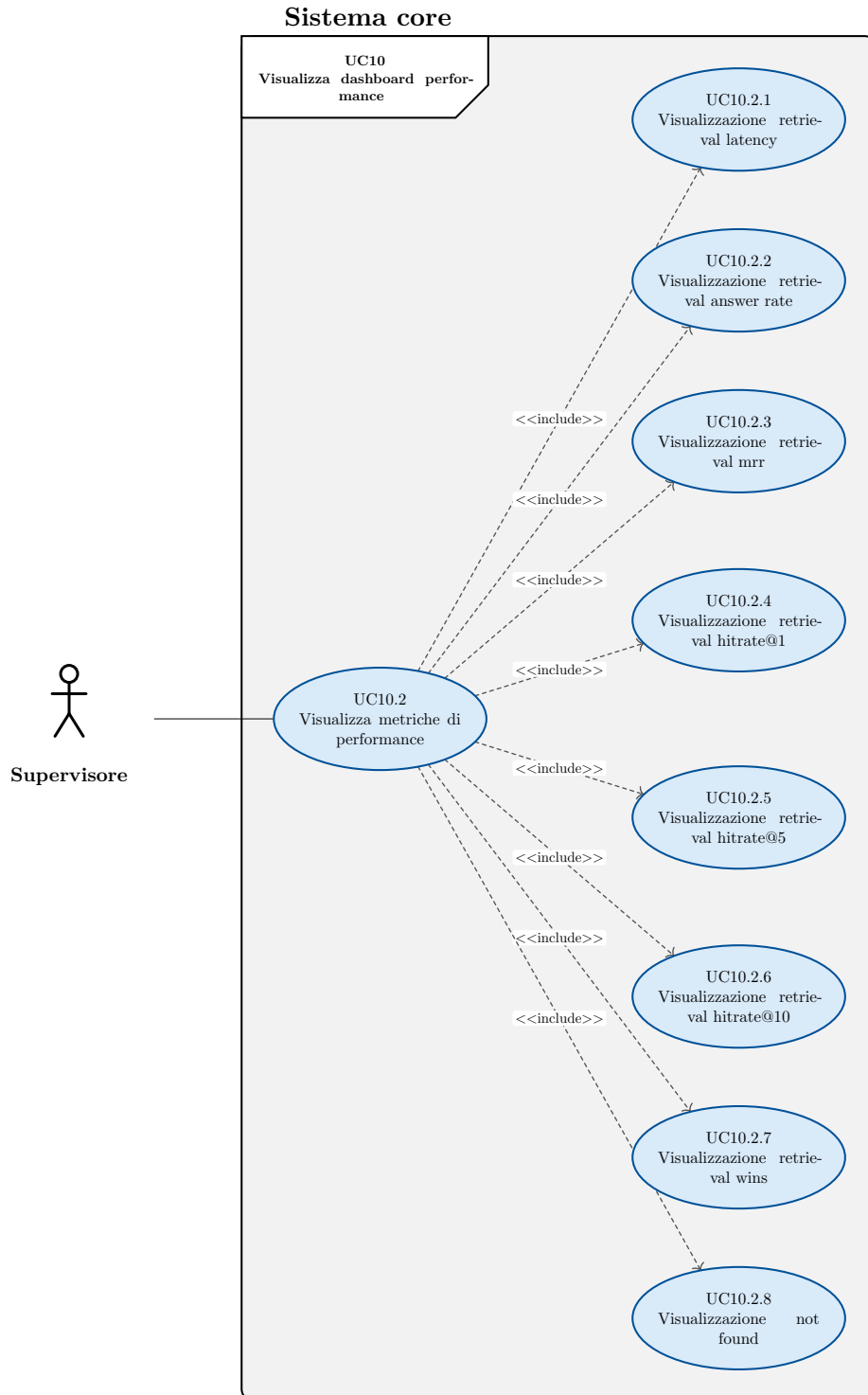


Figura 19: Diagramma del caso d'uso: Visualizza metriche di performance

3 Analisi dei requisiti

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Companion sta visualizzando la dashboard delle metriche
- **Postcondizioni:**
 - Il supervisore ha visualizzato la lista delle metriche
- **Scenario principale:**
 - Il sistema calcola le metriche
 - Il supervisore visualizza l'elenco delle metriche
 - Il supervisore visualizza la retrieval latency → UC10.2.1
 - Il supervisore visualizza la retrieval answer rate → UC10.2.2
 - Il supervisore visualizza la retrieval mrr → UC10.2.3
 - Il supervisore visualizza il retrieval hitrate@1 → UC10.2.4
 - Il supervisore visualizza il retrieval hitrate@5 → UC10.2.5
 - Il supervisore visualizza il retrieval hitrate@10 → UC10.2.6
 - Il supervisore visualizza il retrieval wins → UC10.2.7
 - Il supervisore visualizza il not found → UC10.2.8
- **Trigger:** Il supervisore vuole visualizzare le metriche di performance del sistema di retrieval

UC10.2.1: Visualizzazione retrieval latency

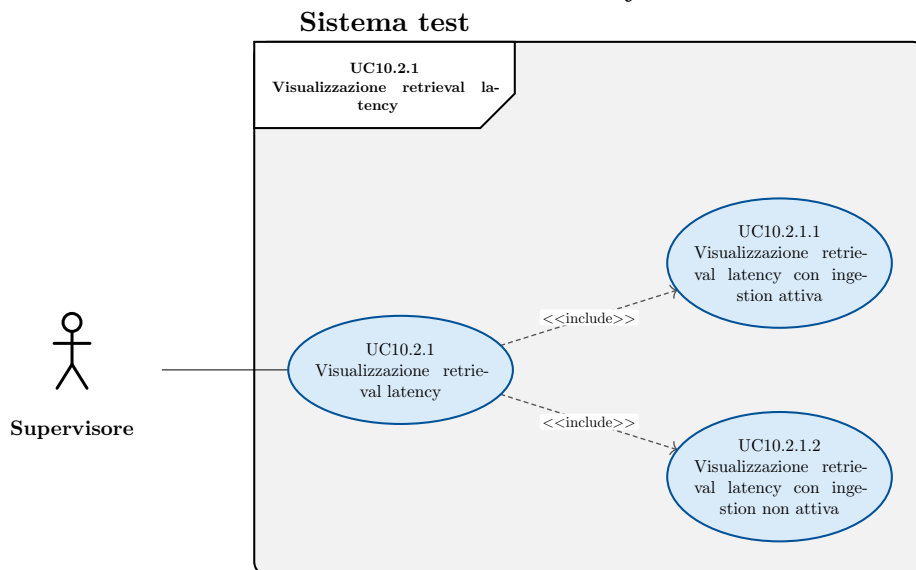


Figura 20: Diagramma del caso d'uso: Visualizzazione retrieval latency

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso

3 Analisi dei requisiti

- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante la presenza di un processo di ingestion → UC10.2.1.1
 2. Il supervisore visualizza la retrieval latency calcolata media durante l'assenza di un processo di ingestion → UC10.2.1.2
- **Inclusioni:**
 - UC10.2.1.1
 - UC10.2.1.2

UC10.2.1.1: Visualizzazione retrieval latency con ingestion attiva

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media durante la presenza di un processo di ingestion
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante la presenza di un processo di ingestion

UC10.2.1.2: Visualizzazione retrieval latency con ingestion non attiva

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media durante la assenza di un processo di ingestion
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante la assenza di un processo di ingestion

UC10.2.2: Visualizzazione retrieval answer rate

- **Attore principale:** Supervisore

3 Analisi dei requisiti

- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval answer rate
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval answer rate

UC10.2.3: Visualizzazione retrieval mrr

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval mean reciprocal rank
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval mean reciprocal rank

UC10.2.4: Visualizzazione retrieval hitrate@1

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@1
- **Scenario principale:**
 - Il supervisore visualizza il retrieval hitrate@1

UC10.2.5: Visualizzazione retrieval hitrate@5

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@5
- **Scenario principale:**
 - Il supervisore visualizza il retrieval hitrate@5

UC10.2.6: Visualizzazione retrieval hitrate@10

- **Attore principale:** Supervisore

3 Analisi dei requisiti

- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@10
- **Scenario principale:**
 - Il supervisore visualizza il retrieval hitrate@10

UC10.2.7: Visualizzazione retrieval wins

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval wins
- **Scenario principale:**
 - Il supervisore ha visualizzato la retrieval wins

UC10.2.8: Visualizzazione not found

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval not found
- **Scenario principale:**
 - Il supervisore visualizza il retrieval retrieval not found

UC10.3: Visualizza metriche di consumo delle risorse

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la dashboard delle performance
- **Postcondizioni:**
 - Il supervisore ha visualizzato le metriche relative al consumo di risorse del DB.
- **Scenario principale:**
 1. Il supervisore visualizza le metriche relative al consumo di risorse del database

UC11: Visualizza dashboard performance test in corso

- **Attore principale:** Supervisore

3 Analisi dei requisiti

- **Precondizioni:**
 - Il sistema è attivo
 - Nel sistema è in corso una run di test
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato in tempo reale dell'ultima run di test avviata
- **Scenario principale:**
 1. Il sistema mantiene aggiornata la dashboard
 2. Il supervisore visualizza l'id della run di test → UC10.1
 3. Il supervisore visualizza la lista delle metriche → UC10.2
- **Scenari alternativi:**
 - Nel sistema non vi sono run di test registrate → UC13

UC12: Visualizza dashboard performance test terminato

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema è attivo
 - Nel sistema non sono in corso run di test
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato dell'ultima run di test avviata
- **Scenario principale:**
 1. Il sistema calcola una singola volta i dati da mostrare
 2. Il supervisore visualizza l'id della run di test → UC10.1
 3. Il supervisore visualizza la lista delle metriche → UC10.2
- **Scenari alternativi:**
 - Nel sistema non vi sono run di test registrate → UC13

UC13: Nessuna run di test trovata

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema è attivo
- **Postcondizioni:**
 - Il supervisore è stato informato dell'assenza di run di test
- **Scenario principale:**
 1. Il sistema non trova nessuna run di test da mostrare
 2. Il supervisore viene informato dell'assenza di run di test su cui calcolare le metriche

3.3 Tracciamento dei requisiti

Ad ogni requisito è associato un codice costruito in base alle sue caratteristiche:

3 Analisi dei requisiti

R(F/Q/C)(M/D/O)

F (*Functional*): definisce una funzione di un sistema o dei suoi componenti;

Q (*Qualitative*): rappresentano come il sistema deve essere per soddisfare i requisiti dello stakeholder;

C (*Constraint*): rappresentano dei vincoli o dei limiti che il sistema deve rispettare;

M (*Mandatory*): irrinunciabili per qualcuno degli stakeholder;

D (*Desirable*): non strettamente necessari ma a valore aggiunto riconoscibile;

O (*Optional*): relativamente utili oppure contrattabili anche in fasi avanzate del progetto;

R (*Requirement*): requisito

In Tabella 1, Tabella 2 e Tabella 3 sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

Codice	Descrizione	Fonti
RFM01	Companion deve poter caricare documenti nel sistema per renderli ricercabili	UC01
RFM02	Companion deve poter esplicitamente avviare una sessione di ingestion dei documenti	UC01.1
RFM03	Companion deve poter caricare liste di documenti, relativi a più tipi di entità, da rendere disponibili per la ricerca	UC01.2
RFM04	Companion deve poter caricare una lista di documenti, relativi a un singolo tipo di entità, da rendere disponibili per la ricerca	UC01.2.1
RFM05	Companion deve poter caricare un numero variabile di documenti in blocco	UC01.2.1.1
RFM06	Companion deve poter caricare una lista di ticket da rendere disponibili per la ricerca	UC01.2.2

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM07	Companion deve poter caricare una lista di conversation item da rendere disponibili per la ricerca	UC01.2.3
RFM08	Companion deve poter caricare una lista di attachments da rendere disponibili per la ricerca	UC01.2.4
RFM09	Companion deve poter indicare esplicitamente la fine della sessione di ingestion dei documenti	UC01.3
RFM10	Companion deve poter essere notificato di errori durante la terminazione della sessione di ingestion	UC01.3.1
RFM11	Companion deve poter essere notificato del fallimento di uno o più record durante il processo di ingestion	UC02
RFM12	Companion deve poter effettuare ricerche basate sulla similarità del testo su una singola entità	UC03
RFM13	Companion deve poter inserire una query di ricerca valida. Una query valida è composta dalle seguenti informazioni obbligatorie: • nome dell'entità su cui eseguire la ricerca, • l'elenco dei campi da ritornare al termine della ricerca, • l'elenco dei campi su cui effettuare la ricerca per similarità, • il testo da usare per la ricerca, • il numero di risultati voluti. Una query valida è composta dalle seguenti informazioni opzionali: • il filtro,	UC03.1

3 Analisi dei requisiti

Codice	Descrizione	Fonti
	- l'elenco dei pesi da applicare ai campi usati nel calcolo della similarità, - l'informazione di lingua opzionale.	
RFM14	Companion deve poter ricevere i risultati della ricerca. Il risultato della ricerca deve contenere le seguenti informazioni: - i valori dei campi richiesti tramite la query, - il nome del campo di testo su cui è stato trovato il match, - il testo su cui è stata rilevata la corrispondenza, - il numero del chunk (si veda i requisiti di vincolo), - il punteggio di similarità.	UC03.2
RFM15	Companion deve poter effettuare ricerche full-text su una singola entità	UC03.3
RFM16	Companion deve poter effettuare ricerche semantiche su una singola entità	UC03.4
RFM17	Companion deve poter effettuare ricerche ibride, basate sia su ricerca full-text sia su ricerca semantica, su una singola entità	UC03.5
RFM18	Companion deve poter inserire dei pesi da utilizzare durante la fusione dei risultati di ricerca da usare al posto dei pesi di default	UC03.5.1
RFM19	Companion deve poter effettuare ricerche ibride, basate sia su ricerca full-text sia su ricerca semantica, su una singola entità. Per la combinazione dei risultati viene usato rrf	UC03.5.2

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM20	Companion deve poter ricevere una notifica di errore in caso di query non valida	UC04
RFM21	Companion deve poter ricevere una notifica di errore in caso di errori durante la ricerca	UC05
RFM22	Companion deve poter eseguire ricerche basate sulla similarità su tutte le entità e combinarne i risultati.	UC06
RFM23	Companion deve poter inserire una query di ricerca linked. La query deve contenere i seguenti dati: • elenco dei campi dati da ritornare indicati tramite nome dell'entità e nome del campo dati, • elenco dei campi dati su cui eseguire la ricerca semantica indicati tramite nome dell'entità e nome del campo dati, • il testo da utilizzare per la ricerca semantica, • il numero di risultati voluti, La query può contenere i seguenti dati: • il filtro da applicare durante la ricerca iniziale, precedente ai join, su ogni entità, • il filtro da applicare successivamente ai join, • i pesi da applicare ai singoli campi su cui calcolare la similarità, • l'informazione di lingua	UC06.1
RFM24	Companion deve ricevere i risultati della ricerca linked. I risultati devono contenere le seguenti informazioni: • i valori dei campi richiesti tramite la query,	UC06.2

3 Analisi dei requisiti

Codice	Descrizione	Fonti
	- il nome dell'entità e il nome del campo di testo su cui è stato trovato il match, - il testo su cui è stata rilevata la corrispondenza, - il numero del chunk (si veda i requisiti di vincolo), - il punteggio di similarità.	
RFM25	Companion deve poter eseguire una ricerca full-text in modalità linked.	UC06.3
RFM26	Companion deve poter eseguire una ricerca semantica in modalità linked.	UC06.4
RFM27	Companion deve poter eseguire una ricerca ibrida in modalità linked.	UC06.5
RFM28	Companion deve poter inserire dei pesi per la fusione di ricerca linked e full-text da usare al posto dei pesi di default	UC06.5.1
RFM29	Companion deve poter effettuare una ricerca ibrida in modalità linked, combinando i risultati tramite rrf	UC06.5.2
RFM30	Companion deve poter ricevere una notifica esplicita in caso venga inserita una query di ricerca linked non valida	UC07
RFM31	Companion deve poter ricevere un messaggio di errore esplicativo in caso di fallimenti nella ricerca linked	UC08
RFM32	Il supervisore deve poter avviare i test di performance del sistema di information retrieval.	UC09

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM33	Il supervisore deve poter visualizzare la dashboard riepilogativa delle performance del sistema	UC10
RFM34	Il supervisore deve poter visualizzare a quale run di test appartengono le metriche presenti sulla dashboard	UC10.1
RFM35	Il supervisore deve poter visualizzare le metriche di performance relative a qualità e prestazioni della ricerca	UC10.2
RFM36	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca	UC10.2.1
RFM37	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca quando è attivo il processo di ingestion dei dati.	UC10.2.1.1
RFM38	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca quando non è attivo il processo di ingestion dei dati.	UC10.2.1.2
RFM39	Il supervisore deve poter visualizzare quanto spesso il risultato atteso è presente tra i risultati reali della ricerca, indipendente dalla posizione.	UC10.2.2
RFM40	Il supervisore deve poter visualizzare il mean reciprocal rank medio delle ricerche	UC10.2.3
RFM41	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente come primo risultato della ricerca.	UC10.2.4

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM42	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente tra i primi 5 risultati della ricerca.	UC10.2.5
RFM43	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente tra i primi 10 risultati della ricerca.	UC10.2.6
RFM44	Il supervisore deve poter visualizzare quante volte la ricerca ha prodotto risultati, indipendentemente dalla loro correttezza	UC10.2.7
RFM45	Il supervisore deve poter visualizzare quante volte la ricerca non ha prodotto risultati	UC10.2.8
RFM46	Il supervisore deve poter visualizzare le performance della run di test attualmente in corso	UC11
RFM47	Il supervisore deve poter visualizzare le performance dell'ultima run di test eseguita	UC12
RFM48	Il supervisore deve poter ricevere notifica esplicita in caso di mancanza di run di test da poter visualizzare	UC13
RFD01	Il supervisore deve poter visualizzare le metriche di consumo delle risorse relative al database	UC10.3
RFO01	Companion deve poter effettuare ricerche ibride in modalità linked combinando i risultati tramite un modello di re-ranking	UC06.5.3
RFO02	Companion deve poter effettuare ricerche ibride su singole entità combinando	UC03.5.3

3 Analisi dei requisiti

Codice	Descrizione	Fonti
	i risultati tramite un modello di re-ranking	

Tabella 1: Requisiti funzionali

Codice	Descrizione	Fonti
RQM01	Il sistema deve superare i benchmark previsti sul seguente volume di dati di 10 000 ticket	Colloqui con il tutor
RQD01	Il sistema deve superare i benchmark previsti sul seguente volume di dati di 100 000 ticket	Colloqui con il tutor
RQO01	Il sistema deve superare i benchmark previsti sul seguente volume di dati di 1 000 000 ticket	colloquio con il tutor

Tabella 2: Requisiti di qualità

Codice	Descrizione	Fonti
RCM01	Il sistema principale e il sistema di test devono essere realizzati con python	Colloquio con il tutor
RCM02	Il sistema principale deve utilizzare fastapi per la realizzazione delle API	Colloquio con il tutor
RCM03	Il sistema principale deve utilizzare un modello di embedding remoto di proprietà dell'azienda	Colloquio con il tutor
XX	Il sistema principale deve utilizzare un database relazionale Postgres.	Piano di lavoro

3 Analisi dei requisiti

Codice	Descrizione	Fonti
	La ricerca semantica va realizzata sfruttando l'estensione pgvector.	
RCM05	Il sistema di test deve utilizzare grafana per la costruzione e gestione della dashboard	Colloqui con il tutor
RCM06	Il sistema principale deve realizzare l'indicizzazione testuale e vettoriale per il modello dati del service desk HDA. Il modello è costituito dalle seguenti entità: • ticket • conversation item • attachment	Piano di lavoro
RCM07	Il sistema principale e il suo modello dati devono essere altamente configurabili. La richiesta di configurabilità è da intendersi nel seguente modo: • le entità che compongono il sistema devono essere configurabili esternamente, • i campi da utilizzare nella ricerca per similarità devono essere configurabili esternamente, • i campi filtrabili devono essere configurabili esternamente, • i pesi da usare nella ricerca devono poter subire override nell'ambito di una singola ricerca • i pesi da usare per la fusione di ricerca semantica e ibrida devono poter subire override nell'ambito di una singola ricerca	Colloquio con i tutor
RCM08	Il sistema principale deve implementare i seguenti tipi di ricerche di similarità sulle singole entità:	Piano di lavoro

3 Analisi dei requisiti

Codice	Descrizione	Fonti
	• Semantica • Full-text • Ibrida	
RCM09	Il sistema principale deve rispettare il seguente comportamento per le ricerche full-text. Se la query di ricerca contiene l'informazione di lingua il sistema limita il pool di candidati solo ai chunk della medesima lingua e utilizza solo la configurazione linguistica assegnata a tale lingua. Altrimenti tutti i chunk vengono valutati sia usando una configurazione di testo agnostica rispetto alla lingua sia usando la configurazione di testo per la lingua assegnata	Colloquio con il tutor
RCM10	Il sistema principale deve implementare per ogni tipo di ricerca di similarità su singola entità anche la rispettiva versione linked. Le sequenze di linking sono le seguenti • Conversation item → Ticket • Attachment → Ticket • Attachment → Conversation item → Ticket	Colloquio con i tutor
RCM11	Il sistema principale deve effettuare ogni tipo di ricerca tramite una singola query interpellando il database una sola volta	colloquio con i tutor
RCM12	Il sistema principale deve implementare la funzionalità di ingestion dei dati	Piano di lavoro

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RCM13	Il sistema di test deve simulare 10 utenti che eseguono 1 query ogni 10 secondi	Colloquio con il tutor
RCD01	Il sistema principale supportare il deployment tramite kubernetes	Colloqui con i tutor
RCD02	Il sistema principale deve supportare flussi di dati paralleli e non ordinati di dati durante l'ingestion. Con non ordinati si intende che è possibile caricare prima gli attachment e poi i ticket.	Colloquio con il tutor
RCO01	Il sistema deve supportare la ricerca ibrida con utilizzo di un modello di re-ranking per la fusione dei risultati.	Piano di lavoro

Tabella 3: Requisiti di vincolo

Di seguito, nella Tabella 4 ho inserito il riepilogo dei requisiti, suddivisi per tipologia e necessità.

Tipo	Mandatory	Desirable	Optional	Somma
Functional	48	1	2	51
Constraint	13	2	1	16
Qualitative	1	1	1	3
Totale	62	4	4	70

Tabella 4: Riepilogo dei requisiti.

Capitolo 4

Introduzione Teorica

In questo capitolo approfondisco le basi teoriche rilevanti per la realizzazione del progetto, le tecnologie rilevanti per il progetto e relativi ruoli nella risoluzione dei problemi affrontati, quali strumenti sono stati adottati e altri strumenti adottati durante lo sviluppo.

4.1 Contesto e problema

Il progetto ha una finalità esplorativa, mira solo a valutare l'adeguatezza di Postgres come sistema di information retrieval.

L'adeguatezza è valutata secondo due criteri: i tempi di risposta e la qualità del retrieval, misurata tramite metriche di hit rate a diversi livelli di granularità. La definizione completa delle metriche è riportata nel caso d'uso UC10.2-Visualizza metriche di performance.

È stato esplicitamente concordato con il tutor aziendale che le metriche relative al consumo di risorse non sono prioritarie per il tirocinio, in quanto una volta effettuato il deployment su server aziendale è già realizzato il tracciamento del consumo di risorse ed è quindi possibile estendere la dashboard Grafana per mostrare anche quello

L'azienda dispone già di un sistema di information retrieval basato su Elasticsearch, tuttavia questo sistema ha delle limitazioni legate alla sua natura non relazionale e orientata ai documenti. Il sistema basato su Postgres mira a replicarne le funzionalità in modo fedele eccetto per alcune deviazioni, indicate nella Capitolo 4.1.1, che rispecchiano quanto realmente voluto dall'azienda ma che non è possibile realizzare nel sistema basato su Elasticsearch

Per giustificare alcune scelte e capire meglio l'utilità di alcune funzionalità, in particolare della ricerca linked, è utile ricordare che il sistema di information retrieval si colloca dentro un sistema RAG, per ulteriori informazioni si veda Capitolo 2

4 Introduzione Teorica

Come già detto nel requisito RCM06 il modello dati di riferimento è quello del ticket service HDA.

Costituito dai seguenti elementi:

- Ticket
- Conversation item
- Attachments

E dalle seguenti relazioni 1 a molti:

- Ticket $\rightarrow$ Conversation item
- Ticket $\rightarrow$ Attachment
- Conversation item $\rightarrow$ Attachment

La ricerca per similarità viene eseguita in due modalità: su singola entità e linked

La ricerca su singola entità esegue la ricerca solo su una delle entità del modello dati mentre la ricerca linked cerca su tutte le unità del modello e ricostruisce per ogni entità cercata una visione globale del risultato tramite join e poi unisce i dati ottenuti in un unico risultato, il funzionamento dettagliato è descritto in TODO CAP PROGETTAZIONE

4.1.1 Limiti di Elasticsearch

Il limite principale di Elasticsearch è la non natività dei join, non nascendo come database relazionale il supporto ai join non è nativo e richiede work-around.

Un altro limite del sistema basato su Elasticsearch è l'impossibilità di eseguire la ricerca per similarità su sottoinsiemi di campi di testo divisi in chunk, questo non è direttamente un limite di Elasticsearch ma deriva anche da come viene usato nell'applicativo aziendale che lo usa. Tale funzionalità però è implementata per la ricerca full-text.

Per rendere più chiaro in cosa consiste questo limite utilizziamo un esempio, è possibile che si voglia effettuare la ricerca semantica solo sul campo dati Problem o solo sul campo Solution di un ticket, con l'attuale sistema basato su Elasticsearch non è possibile.

Per questo motivo il sistema di ricerca realizzato durante il tirocinio non rispecchierà Elasticsearch sotto questo aspetto, invece si allineerà con il comportamento desiderato dall'impresa.

4.2 Basi teoriche

L'*Information retrieval (IR)*_G si occupa di individuare, all'interno di una collezione di dati, gli elementi più pertinenti rispetto a una richiesta espressa dall'utente. Nel contesto di questo progetto la richiesta è rappresentata da una query testuale, mentre la collezione può coincidere

4 Introduzione Teorica

con i dati di una singola entità del modello oppure con l'insieme delle entità collegate secondo le regole di join configurate.

I risultati restituiti da un sistema di IR non costituiscono un insieme non ordinato, ma una lista ordinata secondo un criterio di rilevanza decrescente, detta *ranking*. Questo concetto è alla base delle metriche di valutazione adottate nel progetto, discusse in Capitolo 4.1

Per recuperare i dati da un sistema di information retrieval vengono usate delle funzioni di *similarity-search*.

All'interno del progetto vengono usati i seguenti tipi di ricerca per similarità:

- **ricerca full-text**, usa un criterio di similarità basato sulla corrispondenza di lessemi e parole chiave;
- **ricerca semantica**, usa un criterio di similarità basato sulla distanza dei vettori di embedding corrispondenti alle frasi confrontate;
- **ricerca ibrida**, utilizza sia ricerca semantica sia ricerca full-text e ne combina i risultati con tecniche che ne gestiscono la diversa scala di punteggi.

La ricerca ibrida è la tipologia più rilevante ai fini del progetto. Le altre due assumono invece un ruolo secondario, principalmente di supporto al debugging: valutare le query di ricerca full-text e semantica in isolamento permette di individuare più facilmente la causa di un comportamento inaspettato nella ricerca ibrida, che altrimenti ne combinerebbe gli effetti rendendone più difficile l'analisi.

La ricerca ibrida combina i punti di forza della ricerca semantica e di quella full-text. La ricerca semantica non offre buone prestazioni nel keyword matching, punto di forza della ricerca full-text; quest'ultima, di contro, non è in grado di tracciare termini simili ma con forma testuale molto diversa, né di cogliere significati legati al contesto, aspetti in cui la ricerca semantica eccelle.

La combinazione unisce i risultati di entrambe le ricerche, assegnando un punteggio maggiore (boost) a quelli individuati da entrambe, senza scartare i risultati rilevanti prodotti da una sola delle due.

Questa fusione avviene principalmente tramite Reciprocal Rank Fusion (RRF) (casi d'uso UC03.5.2Ricerca ibrida con RRF e UC06.5.2Ricerca linked ibrida con RRF) oppure tramite modelli di re-ranking (casi d'uso UC03.5.3Ricerca ibrida con modello di re-ranking e UC06.5.3Ricerca linked ibrida con modello di re-ranking).

4.3 Architettura del progetto

Ho scelto di separare il progetto in due sistemi distinti e indipendenti: sistema principale e sistema di test.

4 Introduzione Teorica

Tale separazione garantisce lo sviluppo e la manutenzione indipendenti dei due sistemi.

4.3.1 Sistema principale

Il sistema principale è responsabile dell'implementazione del sistema di information retrieval: offre funzionalità di ingestion dei dati e di ricerca secondo le diverse modalità descritte in Capitolo 4.1.

Segue il pattern dell'architettura esagonale per ridurre il rischio che dei bias modellino il sistema in modo da dare un vantaggio ingiusto a Postgres, come indicato dal rischio R10.

È pensato per l'esecuzione su server.

4.3.2 Sistema di test

Il sistema di test ha il ruolo di eseguire i test di performance della ricerca e calcolare le relative metriche. Simula un numero configurabile di utenti paralleli che inviano richieste di ricerca al sistema principale, confronta i risultati ottenuti con la ground truth attesa, e registra query di test, ground truth e risultati su un database dedicato.

Segue anch'esso il pattern dell'architettura esagonale, per coerenza con il sistema principale e per facilitarne la manutenzione.

È pensato per l'esecuzione locale, sulla macchina da cui viene avviato il test.

4.3.3 Dashboard Grafana

Nel rispetto del requisito RCM05 la dashboard per il controllo delle performance è gestita con Grafana.

Legge il database del sistema di test per calcolare le metriche, gestendo automaticamente il refresh e la selezione della dashboard relativa all'esecuzione di test più recente.

Grafana non fa parte dei sistemi applicativi sviluppati: vengono forniti solamente i file YAML e JSON necessari a costruirla.

4.3.4 Relazione tra i sistemi

I due sistemi non condividono né codice (eccetto un riuso di tipo copia-incolla, per convenienza) né risorse, e sono sviluppati su repository separati.

Il sistema di test comunica con il sistema principale simulando client esterni che inviano richieste di ricerca, mentre Grafana accede direttamente al database del sistema di test, bypassandolo, per leggerne le metriche.

4.4 Criteri di scelta delle tecnologie

I criteri di scelta delle tecnologie sono differenti per i due sistemi, per cui vengono approfonditi separatamente nelle sezioni sistema principale e sistema di test.

4.4.1 Sistema principale

La maggior parte delle tecnologie è fissata dai requisiti di vincolo (Tabella 3). Sono rimaste come scelte libere la libreria di language detection e la scelta del meccanismo di ricerca full-text.

Per la ricerca full-text, oltre alla soluzione nativa di Postgres basata su tsvector e tsquery, sono state valutate alcune estensioni che la implementano tramite l'algoritmo BM25. I criteri richiesti sono: assenza di problemi di licenza compatibili con l'uso in un prodotto commerciale, e un livello di funzionalità sufficientemente avanzato rispetto alle esigenze del progetto.

Un'altra scelta libera riguarda la libreria utilizzata per il riconoscimento della lingua del testo. Il criterio decisivo non è la compatibilità con la versione di Python utilizzata dal progetto (3.14).

Inoltre si è preferito non adottare librerie di query building, per mantenere il massimo controllo possibile sul codice SQL prodotto e non nascondere la complessità, coerentemente con il criterio già seguito per l'architettura del sistema (Capitolo 4.3.1).

4.4.2 Sistema di test

Non sono stati posti vincoli specifici sulle tecnologie adottate: la scelta è stata guidata dalla loro capacità di soddisfare i requisiti, cercando al contempo di non introdurre tecnologie ulteriori rispetto a quelle già usate nel sistema principale, per non aumentare la curva di apprendimento necessaria allo sviluppo.

4.5 Tecnologie

Nelle seguenti sezioni viene analizzato l'insieme di tecnologie adottate per la realizzazione del progetto.

Ogni tecnologia è contrassegnata anche dal relativo numero di versione, in quanto vi potrebbero essere stati aggiornamenti significativi che rendono obsolete considerazioni tecniche presenti in questo documento. Il progetto è composto da 2 sistemi distinti e indipendenti, perciò i relativi stack tecnologici sono analizzati separatamente nelle sezioni sistema principale e sistema di test.

4 Introduzione Teorica

Fanno eccezione Python, Poetry e Postgres, in quanto comuni ad entrambi gli stack tecnologici, mentre Grafana non fa formalmente parte di nessuno dei 2 sistemi.

Python v3.14.X

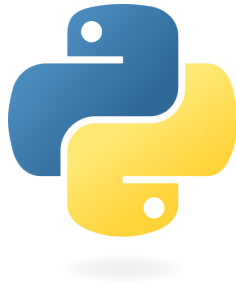


Figura 21: Logo Python

- **Descrizione:** Linguaggio di programmazione ad alto livello che supporta diversi paradigmi di programmazione.
- **Motivazione della scelta:** Richiesto dal requisito RCM01

Poetry v2.0.0

- **Descrizione:** Strumento per la gestione delle dipendenze
- **Motivazione della scelta:** Scelto perché è uno strumento che ho già usato in passato

PostgreSQL v17.4.0

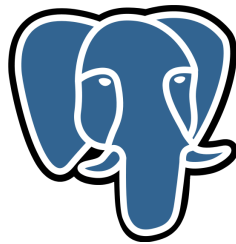


Figura 22: Logo Postgres

- **Descrizione:** Comunemente noto come **Postgres**, è un database open source che vanta una solida reputazione in termini di affidabilità, flessibilità e supporto degli standard tecnici aperti.
- **Motivazione della scelta:** Richiesto dal requisito XX

4 Introduzione Teorica

Grafana v13.2.0



Figura 23: Logo Grafana

- **Descrizione:** Piattaforma software open source per la visualizzazione, l'analisi e il monitoraggio dei dati.
- **Motivazione della scelta:** Richiesto dal requisito RCM05

4.5.1 Sistema principale

Le tecnologie adottate all'interno del sistema principale sono divise come segue.

4.5.1.1 Backend

Fastapi v0.115.0

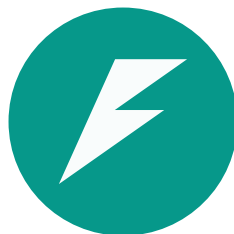


Figura 24: Logo Fastapi

- **Descrizione:** Framework web veloce e moderno per la costruzione di api con Python.
- **Motivazione della scelta:** Richiesto dal requisito RCM02 . Supporto nativo per l'asincronia.

Asyncpg v0.31.0

- **Descrizione:** Libreria python dedicata alla comunicazione con database Postgres in un contesto python asincrono.
- **Motivazione della scelta:** Scelta per via della sua efficienza e per il supporto all'asincronia.
- **Alternative valutate:**
 - **SQLAlchemy v2.0.0:** è un ORM, quindi è stato scartato in accordo con i criteri di scelta delle tecnologie stabiliti in Capitolo 4.4.1

Pgvector-python v0.5.0

- **Descrizione:** Libreria dedicata al supporto di pgvector in python.
- **Motivazione della scelta:** Necessaria per permettere a Asyncpg di utilizzare funzioni e tipi di pgvector.

Py3langid v0.3.0

- **Descrizione:** Libreria dedicata alla language detection.
- **Motivazione della scelta:** Scelta per la compatibilità con Python 3.14
- **Alternative valutate:**
 - **fasttext-langdetect v1.1.1:** minore manutenzione rispetto alla libreria scelta, incompatibile con python 3.14

4.5.1.2 Database

PostgreSQL Full-Text Search v17.4.0

- **Descrizione:** Meccanismo di ricerca full-text nativo di Postgres, basato sui tipi di dato tsvector e tsquery e sulla funzione di ranking ts_rank.
- **Motivazione della scelta:** Nessun problema di licenza legato all'uso in un prodotto commerciale, e livello di funzionalità sufficientemente avanzato rispetto alle esigenze del progetto. Vi sono comunque presenti lacune di funzionalità che hanno richiesto dei workaround analizzati nel dettaglio in [TODO LINK ALLA SEZIONE LAVORO SVOLTO RICERCA FULL-TEXT](#).

Il limite tecnologico principale è dato dalla non implementazione della funzione di ranking bm25, il testo viene valutato solo sulla base del testo della ts_query e del testo del ts_vector, senza utilizzo di un corpus di documenti per ripesare il punteggio del testo.

Sebbene questo possa penalizzare la qualità del ranking, garantisce che i punteggi calcolati su entità diverse siano direttamente comparabili.

Limite accettato dato che lo scopo del progetto è valutare la qualità della ricerca, con focus principale su pgvector.

- **Alternative valutate:**
 - **ParadeDB** v0.25.0: licenza incompatibile con i vincoli aziendali sull'uso commerciale
 - **pg_textsearch (TimescaleDB)** v1.3.1: il rapporto tra vantaggi, limitazioni e incognite da valutare non è stato ritenuto sufficientemente alto da giustificare l'adozione, per le limitazioni si veda https://github.com/timescale/pg_textsearch/blob/v1.3.1/README.md#limitations, le più importanti sono la mancanza di phrase_query, l'incognita di comparabilità dei punteggi di diverse partizioni di una tabella, il partizionamento era già previsto per conformarsi raccomandazioni di pg vector, la stessa incognita va applicata alla confrontabilità dei punteggi di 2 entità diverse durante la ricerca linked

Pgvector v0.8.4

- **Descrizione:** Estensione Postgres che implementa funzionalità di ricerca semantica basate sui vettori
- **Motivazione della scelta:** Richiesto dal requisito XX

4.5.1.3 Deployment

Il sistema principale è progettato per essere containerizzato tramite Docker, requisito necessario per il successivo deployment su Kubernetes

nell'infrastruttura aziendale. Il deployment effettivo su Kubernetes, così come le valutazioni condotte sui dati e sui database aziendali reali, non rientrano nel lavoro svolto durante il tirocinio: sono demandati al team aziendale competente, sia per ragioni organizzative sia per assenza delle credenziali necessarie ad accedervi.

Ai fini dello sviluppo e del test in locale è stato realizzato il Dockerfile per il sistema, insieme a una configurazione Docker Compose completa che include anche i servizi di supporto necessari (ad esempio Postgres). Docker Compose orchestra localmente gli stessi container Docker utilizzati poi in ambiente Kubernetes, garantendo coerenza di comportamento tra ambiente di sviluppo e ambiente di produzione.

4.5.1.4 Strumenti di supporto

Altri strumenti di supporto degni di nota sono **uvicorn**, **pydantic-settings**, **sqlparse**.

- Uvicorn è un server ASGI, scelta strettamente legata all'uso di Fastapi.
- Pydantic-settings viene usato per avere una gestione più pulita delle variabili d'ambiente, non richiede l'aggiunta di ulteriori dipendenze in quanto Fastapi utilizza Pydantic
- Sqlparse è una libreria di parsing e formattazione SQL, usata per migliorare la leggibilità delle query ricostruite durante il debug e il logging

4.5.2 Sistema di test

Le tecnologie adottate all'interno del sistema di test sono divise come segue.

4.5.2.1 Backend

Locust v2.32.0

- **Descrizione:** Strumento per il testing di funzionalità che coinvolgono chiamate con protocollo http o altri protocolli
- **Motivazione della scelta:** Scelto per la sua semplicità, adeguatezza al caso d'uso di simulazione di vari utenti paralleli e per la sua estensibilità

Httpx v0.28.0

- **Descrizione:** Strumento per la comunicazione http
- **Motivazione della scelta:** Scelto perchè il più utilizzato, si integra bene con locust

Psycopg v3.0.0

- **Descrizione:** Driver per la comunicazione con un database postgres
- **Motivazione della scelta:** Scelto per la solidità e la compatibilità con locust
- **Alternative valutate:**
 - **Asyncpg** v0.31.0: scartato per la bassa compatibilità con locust che lavora meglio su un contesto python sincrono

4.5.2.2 Database

Vengono usati 2 storage:

- un file jsonl per la persistenza delle query da eseguire e la ground truth, scelta dovuta alla semplicità di condivisione e alla mancanza di requisiti di prestazioni di scrittura e di lettura non sequenziale;
- un database postgres, usato per loggare i risultati dei test, scelto per la facilità di integrazione con Grafana e per la necessità di scritture continue e ricalcolo continuo delle metriche, realizzato con una vista.

Per maggiori dettagli si vedano le informazioni di Postgres

4.5.2.3 Deployment

Anche il sistema di test è containerizzato tramite Docker ed è orchestrato, insieme a database delle metriche e Grafana, tramite un'unica configurazione Docker Compose.

La containerizzazione di Locust non risponde a un'esigenza tecnica stretta: durante lo sviluppo è stato utilizzato esclusivamente in modalità headless, e potrebbe quindi essere eseguito direttamente sulla macchina host. È stata comunque scelta per uniformità con il resto dello stack e per poter avviare l'intero ambiente di test con un unico comando, senza dover gestire manualmente le dipendenze.

Vi è inoltre il vantaggio di un'evoluzione più semplice qualora in futuro si volesse utilizzare l'interfaccia UI offerta da Locust.

4.5.2.4 Strumenti di supporto

Viene usato pydantic-settings per semplificare la gestione delle variabili d'ambiente

4 Introduzione Teorica

Capitolo 5

Analisi iniziale

In questo capitolo descrivo i primi approcci e le prime idee applicate allo svolgimento del progetto.

5 Analisi iniziale

Capitolo 6

Descrizione del Lavoro Svolto

In questo capitolo approfondisco le problematiche affrontate nel concreto e le relative soluzioni.

6.1 Struttura del database

6.1.1 Tabelle

Il database ha la seguente struttura:

- Per ogni entità vengono create le seguenti tabelle:
 - Tabella principale, contiene i dati dell'entità eccetto quelli `CHUNKED_TEXT + SEARCHABLE` (data la loro natura a chunk e la necessità di ottimizzarle per le ricerche full text e semantica richiedono una tabella apposita di supporto), chiavi primarie e chiavi esterne vengono create in accordo con la `SchemaConfiguration`
 - Tabella dei chunk, funge da supporto per le ricerche, contiene i seguenti dati:
 - Chive primaria della tabella principale, funge anche da chiave esterna
 - `field_name`, il nome del campo dati `CHUNKED_TEXT + SEARCHABLE`, necessario a supportare la ricerca di similarità su sotto insiemi, chiave esterna verso un'apposita tabella, non necessaria ma utile a garantire integrità
 - numero del chunk, intero che indica il numero del chunk
 - lingua del testo, chiave esterna verso una tabella globale
 - testo del chunk
 - il vettore di embedding calcolato sul testo del chunk
 - `tsv_simple`, il tsvector calcolato automaticamente dal database dal testo del chunk sfruttando una configurazione agnostica rispetto alla lingua

6 Descrizione del Lavoro Svolto

- `tsv_lang`, il `tsvector` calcolato automaticamente dal database dal testo del chunk sfruttando una configurazione specifica per la lingua indicata nel campo di lingua
- campi `filterable` denormalizzati
- chiave primaria formata dalla chiave esterna verso l'entità di origine, `field_name`, numero del chunk
- tabella dei `field_name`, utile ma non obbligatoria
- tabella delle lingue, utile a fornire vincoli sul supporto linguistico.
- ingestion sessions, tabella usata per tracciare la presenza e l'attività di una sessione di ingestion
- session streams, tabella che traccia l'esecuzione dei singoli stream di inserimento, serve a non far terminare l'ingestion se degli stream stanno ancora eseguendo

6.1.2 Staging

All'interno del database sono anche presenti delle tabelle di staging necessarie a gestire il probabile non arrivo in ordine dei dati

Le tabelle di staging sono relative sia alle entità che ai loro chunk, non viene usata la vera chiave primaria ma uno staging id che permette di gestire conflitti in caso di caricamento duplicato di valori, la chiave primaria della tabella originale rimane sottoposta a vincolo `unique` a sua volta indicizzato per facilitare i join per la fase di promozione sono più semplici rispetto alle loro controparti reali, le altre differenze sono le seguenti:

- non hanno la materializzazione dei `ts vector`,
- i vettori di embedding non sono di tipo `vector` ma `double array`, questo permette di non registrare l'estensione `vector` dentro il pool di connessioni dedicato all'ingestion dei dati, rendendola più leggera, (il casting è automatico in fase di promozione)
- i non vi è denormalizzazione dei campi `filterable`
- non vi è alcuna indicizzazione oltre al `btree` sulle chiavi primarie reali

6.1.2.1 promozione

la promozione avviene in batch di dimensione configurabile tramite query che selezionano ciò che può essere promosso, lo scheduling viene calcolato dinamicamente basandosi sui vincoli di integrità referenziale descritti nello schema configuration

6 Descrizione del Lavoro Svolto

```
1  WITH staging_window AS (SQL
2      SELECT staging_id FROM {source_table}
3      WHERE {where_clause}
4      ORDER BY staging_id
5      LIMIT {self._batch_size}
6  ),
7  batch AS (  
8      SELECT DISTINCT ON ({conflict_columns}) s.staging_id
9      FROM {source_table} s
10     JOIN staging_window w ON s.staging_id = w.staging_id
11     ORDER BY {conflict_columns}, s.staging_id DESC
12  ),
13  promoted AS (  
14     DELETE FROM {source_table}
15     WHERE staging_id IN (SELECT staging_id FROM staging_window)
16     RETURNING staging_id, {column_list}
17  ),
18  inserted AS (  
19     INSERT INTO {target_table} ({column_list})
20     SELECT {column_list} FROM promoted
21     WHERE staging_id IN (SELECT staging_id FROM batch)
22     ON CONFLICT ({conflict_columns}) {conflict_action}
23     RETURNING 1
24  )
25  SELECT count(*) FROM staging_window
```

6.1.3 Partitioning

le tabelle dei chunk sono partizionate sul campo fieldname, questo è fondamentale al supporto efficiente della ricerca per sotto insiemi

6.1.4 Indici

Nell'analisi degli indici vengono omessi gli indici creati in modo autonomo da Postgres in corrispondenza di chiavi primarie va comunque ricordato che essi esistono

le seguenti tabelle hanno i seguenti indici:

- ▶ Tabelle dei chunk:
 - Indice vettoriale hnsw, va impostato sul tipo di vettore e sul tipo distanza che si desidera utilizzare in fase di oversampling

6 Descrizione del Lavoro Svolto

Pgvector raccomanda che le ricerche che fanno uso di indici abbiano un parametro di oversampling, ciò è dovuto alla natura di indici approssimati di hnsw e ivfflat.

un'altra raccomandazione è utilizzare gli indici composti per indicizzare una versione a precisione ridotta dei vettori e utilizzare il re-ranking sui valori esatti

- Indici btree su tutti i campi denormalizzati, questo ha la funzionalità di mitigare i problemi di filtering associati agli indici approssimati vettoriali, se una query con filtro è particolarmente selettiva è possibile che il planner di Postgres decida di applicare prima il filtro e poi fare un calcolo della similarità sui risultati ottenuti, senza l'indice Postgres avrebbe prima recuperato i risultati e poi applicato il filtro, potenzialmente perdendo tutti i risultati, per i filtri poco selettivi invece l'oversampling e iterative scan mitigano la perdita di risultati
- Indici testuali GIN, sulle colonne ts vector vengono creati degli indici testuali per il filtering durante la ricerca full text
 - su tsv simple viene costruito un normale indice gin normale
 - su tsv lang viene costruito un indice parziale per lingua
 - su tsv simple viene costruito un indice gin con l'inclusione di array ops

```
1 CREATE INDEX ... ON ... USING GIN
  ((tsvector_to_array(tsv_simple)) array_ops)
```

 SQL

- su tsv lang viene costruito un indice parziale per lingua con l'inclusione di array ops

vi è una duplicazione sugli indici ts vector a causa di una funzionalità che va supportata ma che la full text nativa non supporta, ricerca con matching di solo un tot di parole per tale ricerca è necessario considerare i tsvector come semplici array di testo, entrambi gli indici sono utilizzati solo in fase di filtering, quindi solo un tipo di indici rimane necessario il ranking rimane affidato alla full text search di Postgres che non necessita comunque dell'indice.

- tabella delle ingestion session
 - unique index sui valori dello status delle sessioni di ingestion per garantire solo una sessione attiva alla volta

```
1 CREATE UNIQUE INDEX
  ix_ingestion_sessions_single_open ON
  public.ingestion_sessions USING btree (status)
  WHERE (status = 'open'::text)
```

 SQL

6.2 Struttura generale del sistema

Il progetto si divide in 2 sistemi separati

- Retriever, il sistema principale che implementa le logiche di ricerca e ingestion dei dati
- Retriever-trial, sistema di test, si interfaccia al sistema principale usando locust per simulare diversi utenti che eseguono query, poi logga i risultati della ricerca su un db che tramite una view calcola le metriche

6.2.1 Retriever

Segue il principio dell'architettura esagonale, questo permette di modellare il sistema basandosi solo sul problema senza modellare funzioni non utili in questa fase esplorativa e che potrebbero andare a lodare Postgres per via di un bias

le porte sono realizzate con ereditarietà da ABC e abstract method

6.2.1.1 Struttura della pipeline di ingestion

La pipeline di ingestion segue il principio dell'architettura esagonale

1. 4 adapter inbound:

- start ingestion session controller
- ingest batch list controller
- end ingestion session controller
- is ingestion active

hanno tutti una porta inbound specifica (suffisso use case) per lo scopo che devono realizzare La serializzazione e deserializzazione è in automatico con pydantic

l'adapter inbound per l'ingestion è realizzato con fastapi, prende in input un array di raw batch un'entità che rappresenta un blocco di dati grezzi in input ritorna una streaming response che comunica su quali record è fallito il processo di ingestion

inizialmente è stata progettata per avere uno stream sia in input che in output ma fastapi non supporta tale funzione, perciò il passaggio a lista di blocchi è stato un fix di convenienza

2. per le 4 porte use case vi sono 4 servizi

i service di avvio, termine e tracciamento di una sessione di ingestion utilizzano una porta outbound ingestion status tracker per realizzare le loro funzionalità

il service di ingestion vero e proprio accetta un async iterator di raw entity batch e usa una porta outbound per tracciare lo status dell'esecuzione dello stream

6 Descrizione del Lavoro Svolto

per l'elaborazione dati vera e propria si avvale di 2 classi helper iniettate alla costruzione, un validator che controlla la correttezza dei dati e si occupa anche di fare casting di tipi non deducibili nell'adapter nell'adapter inbound il tipo di un dato viene dedotto dal suo formato senza richiedere che tale informazione arrivi esplicitamente, tuttavia per i date time la conversione da string a datetime può essere ambigua (se un campo dati che deve rimanere testo ha come valore una data che però deve rimanere come stringa genererebbe errori più avanti, quindi il service durante la validazione dei tipi applica questa trasformazione dove necessario guardando i dati della schema configuration), poi vi è un processor che riceve alla costruzione le porte outbound per la language detection e per il calcolo degli embedding. si occupa di aggregare le liste di testi da passare alle porte e costruire i refined entity batch

infine scrive il refined entity batch tramite una porta write repository il metodo che realizza tutto ciò accetta in input un async iterator e manda in output un async iterator di rejected record comprensivi di identificativo e ragione del fallimento

3. le porte outbound sono quelle :

- per la gestione del tracking della sessione
- per la gestione del tracking degli stream
- per il calcolo degli embedding (accetta una lista di stringhe per poter applicare ottimizzazioni lato adapter)
- per il rilevamento della lingua (accetta una lista di stringhe per poter applicare ottimizzazioni lato adapter)
- per il salvataggio dei dati su db

4. gli adapter sono gli unici a conoscere il db Postgres, ricevono il connection pool alla creazione

gli adapter per il tracciamento della sessione usano le tabelle per loggare lo stato, una sessione di ingestion può avere tre stati aperta chiusa e finalizzata, da aperta a chiusa vuol dire che si possono ricevere richieste di inserimento dati e non si possono aprire altre sessioni, da closed a finalized non è più possibile caricare dati ma non si possono avviare nuove sessioni

la write repository scrive sulle tabelle di staging (sia tabelle principali che tabelle dei chunk), utilizza copy invece di insert per poter eseguire l'operazione in modo più veloce, reso possibile tramite lo staging id auto increment

quando arriva la notifica di termine dell'ingestion viene assegnato lo stato closed e avviato il processo di promozione, il processo di promozione avviene come descritto prima, al suo termine viene assegnato lo stato finalized

6 Descrizione del Lavoro Svolto

il processo di promozione avviene a batch di dimensione configurabile alla costruzione del sistema

6.2.2 Progettazione della ricerca

viene sempre seguita l'architettura esagonale

si usano delle classi di dominio per rappresentare le informazioni necessarie all'esecuzione

```
1 Python  
2     #rappresenta una query generale  
3 @dataclass(frozen=True, slots=True)  
4 class Query(Generic[R]):  
5     #valori da inserire nella clausola select  
6     return_fields: frozenset[R]  
7     #serve a costruire parte della clausola where  
8     filter: FilterCondition[R] | None = None  
9  
10  
11
```

```
1 Python  
2     #arricchisce la query con le informazioni per la ricerca  
3     #semantica  
4 @dataclass(frozen=True, slots=True)  
5 class SimilarityQuery(Generic[R]):  
6     query: Query[R]  
7     target_similarity_fields: frozenset[R]  
8     similarity_search_text: str  
9     field_weights: _Weights[R] | None = None #overridable  
10    language: LanguageCode | None = None #opzionale  
11    top_k: int = 20 #overridable  
12
```

questa è la classe con cui gli adapter inbound inviano richieste alle porte inbound

6 Descrizione del Lavoro Svolto

```
1 Python
2 #wrapper della similarity query arricchito con le info per la
  fusione di ibrida e semantica
3 @dataclass(frozen=True, slots=True)
4 class HybridSearchRequest(Generic[R]):
5
6
7     query: SimilarityQuery[R]
8     merge_weights: MergeWeights | None = None #overridable
9
10
```

```
1 Python
2 #Risultato base di una query
3 @dataclass(frozen=True, slots=True)
4 class QueryResult(Generic[R]):
5
6     results: tuple[Mapping[R, FieldValue], ...]
7
8
```

```
1 Python
2 #info aggiuntive legate alla ricerca semantica
3 @dataclass(frozen=True, slots=True)
4 class SimilaritySearchResult(Generic[R]):
5     query_result: QueryResult[R]
6     matched_field: R
7     chunk_counter: int
8     matched_text: str
9     score: float
10
11 # wrapper creato per comodità
12 class SimilaritySearchResults(Generic[R]):
13
14     results: tuple[SimilaritySearchResult[R], ...]
15
```

6 Descrizione del Lavoro Svolto

```
1 Python
2
3     #wrapper da usare per la porta outbound che esegue la
4     ricerca full text
5 @dataclass(frozen=True, slots=True)
6 class FullTextQueryCommand(Generic[R]):
7
8     query: SimilarityQuery[R]
9
10
```

```
1 Python
2
3 @dataclass(frozen=True, slots=True)
4 class HybridQueryCommand(Generic[R]): #wrapper da usare per
5     la porta outbound che esegue la ricerca ibrida
6     query: SimilarityQuery[R]
7     embedding: VectorEmbedding
8     merge_weights: MergeWeights
9
```

```
1 Python
2 class HybridQueryCommand(Generic[R]): #wrapper da usare per
3     la porta outbound che esegue la ricerca semantica
4 @dataclass(frozen=True, slots=True)
5 class SemanticQueryCommand(Generic[R]):
6
7     query: SimilarityQuery[R]
8     embedding: VectorEmbedding
9
10
```

viene utilizzato un generic solo perché all'inizio le 2 gerarchie linked e hybrid erano identiche e quindi un alias per le versioni con field name e field reference era sufficiente successivamente disallineati lato query per

6 Descrizione del Lavoro Svolto

necessità di specificare dei filtri sia per la singola entità che per la ricerca linked nel complesso
sono ancora allineate sui risultati

6.2.2.1 Progettazione della ricerca semantica su singola entità

la progettazione delle query semantiche si divide su 2 livelli
classi che gestiscono la corrispondenza tra alcuni concetti della ricerca semantica e pg vector e classi che costruiscono concretamente la query.

```
1  
2   #classe base usata per buildare la query  
3   sql: str = ""  
4   params: tuple[Any, ...] = ()  
5  
6
```

permette di concatenare 2 frammenti di query costruiti separatamente mantenendo corretti legami tra i segnaposto e i parametri
ho preferito non usare query builder per assicurare che la complessità della pipeline di ricerca rimanga in chiaro e che ci sia un buon controllo sulle ottimizzazioni

6 Descrizione del Lavoro Svolto

```
1 Python  
2 class PgVectorDistances(StrEnum):  
3     """Basata sul principio 'lower raw distance = more  
4     similar', vero per  
5     tutte le distanze in questo spazio raw nativo di  
6     pgvector.  
7     Per COSINE e INNER_PRODUCT esiste inoltre uno spazio di  
8     similarità  
9     limitato e naturale per chi chiama (higher = more  
10    similar): pgvector  
11    calcola il raw in forma invertita/negata per ottimizzare  
12    l'ordinamento,  
13    quindi va convertito prima di essere esposto o  
14    confrontato con una  
15    soglia espressa in termini di similarità.  
16  
17    Per L2, L1, HAMMING, JACCARD non esiste un simile spazio  
18    limitato: sono  
19    distanze intrinsecamente illimitate, lower=better è la  
20    loro unica forma  
21    sensata. Chi chiama per queste distanze esprime SEMPRE la  
22    soglia (e  
23    riceve SEMPRE lo score) nello stesso spazio raw -  
24    to_raw_threshold e  
25    to_public_score sono no-op per questi casi, non  
conversioni.  
"""  
L2 = auto()  
INNER_PRODUCT = auto()  
COSINE = auto()  
L1 = auto()  
HAMMING = auto()  
JACCARD = auto()  
  
  

```

6 Descrizione del Lavoro Svolto

```
1
2
3 class PgVectorType(StrEnum):
4     VECTOR=auto()
5     HALFVEC=auto()
6     BIT=auto()
7     # SPARSEVEC=auto() #il sistema non lo usa e non è neanche
      possibile il casting
8
9
10
```

6 Descrizione del Lavoro Svolto

```
1 Python
2
3
4 @dataclass(frozen=True, slots=True)
5 class SemanticSimilarityThreshold:
6     """Soglia di attivazione per ranking semantico, espressa
7     in spazio
8     'normalizzato' (higher = more similar) insieme alla
9     distanza per cui
10     è stata pensata - vedi nota su PgVectorDistances sul
11     perché i due non
12     vanno mai disaccoppiati.
13
14     L'operatore in spazio raw è sempre <=: sia
15     normalized_distance_expression
16     per INNER_PRODUCT (-1*raw) sia per COSINE (1-raw) sono
17     trasformazioni
18     monotone DECRESCENTI di raw, quindi 'normalized >=
19     soglia' si riduce
20     sempre a 'raw <= soglia_convertita', qualunque delle due
21     distanze.
22     Non è un dettaglio da iniettare: è conseguenza algebrica
23     della
24     trasformazione stessa.
25     """
26     distance: PgVectorDistances
27     value: float
```

queste classi servono a coniugare il comportamento di pgvector in un modo più coerente con le metriche andando a eseguire le trasformazioni inverse alle ottimizzazioni applicate durante la ricerca per la costruzione della query vera e propria si procede prima eseguendo le query sulle singole partizioni della tabella basandosi sui field name, vengono materializzate con cte e poi unite tramite union all e viene fatto il rescoring, union all e rescoring sono corretti perchè tutti i vettori usano la stessa distanza e i vettori vengono tutti dallo stesso modello di embedding per rendere più facilmente configurabile il sistema la descrizione della configurazione di pg vector è iniettata tramite dependency injection di un oggetto apposito

6 Descrizione del Lavoro Svolto

```
1 Python
2 @dataclass(frozen=True, slots=True)
3 class PgVectorEngineConfig:
4     distance: PgVectorDistances
5     vector_type: PgVectorType
6     user_threshold: float
7     oversampling: int
8     embedding_dimensions: int
9     oversampling_distance: PgVectorDistances | None = None
10
```

il progetto adotta la seguente configurazione

```
1 Python
2     PgVectorEngineConfig(
3         distance= PgVectorDistances.INNER_PRODUCT,
4         vector_type= PgVectorType.VECTOR,
5         user_threshold=0.3,
6         oversampling=20,
7         oversampling_vector_type=PgVectorType.BIT,
8         oversampling_distance=PgVectorDistances.HAMMING,
9         embedding_dimensions=768
10     )
11
12
```

per modificarla è sufficiente modificare una factory apposita che ostruisce tutti i vari oggetti di configurazione, questa gestisce in automatico le variabili ambientali anche se non sono stati utilizzati if per la configurazione l'integrazione di questa funzione è veloce e avviene come modifica a un singolo metodo di una singola classe
l'sql così generato ha circa la seguente forma

6 Descrizione del Lavoro Svolto

```

1
2 WITH candidates AS MATERIALIZED (
3     SELECT
4         "id_7a3c",                                -- PK
5         fisica (ID)
6         'Problem' AS "field_name",                -- nome
7         logico del campo cercato
8         "chunk_text" AS matched_text,
9
10        -- fase 1: candidati generati con cast leggero
11        (oversampling)
12        -- BIT + HAMMING, non a piena precisione
13        binary_quantize("embedding"::vector)::bit(768)
14        <~>
15        binary_quantize(:query_embedding::vector)::bit(768)
16        AS candidate_score,
17
18        -- fase 2: rescoring a piena precisione (VECTOR +
19        INNER_PRODUCT),
20        -- calcolato SOLO sui candidati già scremati dal
21        LIMIT sotto
22        "embedding" <#> :query_embedding AS raw_score,
23        "chunk_counter"
24 FROM tk_chunks_9f21
25 WHERE "field_name" = 'problem_ab12'              --
26        nome FISICO del campo (per il filtro)
27        AND "status_id_44b1" = :ticket_status     --
28        filtro di dominio opzionale (es. 'OPEN')
29 ORDER BY candidate_score ASC                    --
30        ordina sul cast leggero
31 LIMIT :oversampling_limit                      --
32        es. 30 (top_k=10 + oversampling=20)
33 )
34 SELECT
35     "id_7a3c", "field_name", chunk_counter, matched_text,
36     raw_score,
37     1.0::float8 AS field_weight                --
38     peso del campo, noto a build-time
39 FROM candidates
40 WHERE raw_score <= :raw_threshold              --
41     0.3 utente -> -0.3 raw (INNER_PRODUCT invertito)
42
43
44

```

6 Descrizione del Lavoro Svolto

```
1
2 WITH ranked AS MATERIALIZED (
3     SELECT
4         "id_7a3c", "field_name", "matched_text",
5         "chunk_counter", "raw_score",
6         (-1 * ("raw_score")) * "field_weight" AS
7         weighted_score
8     FROM (
9         ( /* partizione "Problem", vedi sopra */ )
10        UNION ALL
11        ( /* stessa struttura, campo "Solution" */ )
12        UNION ALL
13        ( /* stessa struttura, campo "Subject" */ )
14    ) AS all_candidates
15    ORDER BY weighted_score DESC          -- INNER_PRODUCT:
16    higher = più simile
17    LIMIT :top_k                          -- es. 10
18 )
19 SELECT
20     main."created_at_5c10" AS "CreationDate",
21     main."subject_2e9f"   AS "Subject",
22     main."status_id_44b1" AS "TicketStatusID",
23     ranked."field_name", ranked."matched_text",
24     ranked."chunk_counter", ranked."weighted_score"
25 FROM ranked
26 JOIN tk_main_9f21 AS main
27     ON main."id_7a3c" = ranked."id_7a3c"    -- JOIN una sola
28     volta, solo sulle righe già a top_k
29 ORDER BY ranked."weighted_score" DESC
```

```
1
2
3
4
```

le partizioni singole eseguono internamente il reranking e materializzano il loro punteggio moltiplicato per il peso assegnato così la query esterna si limita a fare un union all order by

6.3 Progettazione della ricerca full text

la ricerca full text ha adottato un'approccio simile alla semantica, viene mantenuta la struttura a partition queries perchè servono a mantenere semplice la ricerca su sotto insiemi di field name e ad applicare facilmente i pesi, in questo caso questa forma non è vincolante ma sta più un riuso del codice della ricerca semantica la ranking function nativa di Postgres per la ricerca full text non ha come in elastic search la funzione di boosting progressivo, ad esempio non è possibile dire alla ranking function di dare un boost grande se è un match su frase uno piccolo se è un match su tutte le parole ma non in ordine e poco se vi sono corrispondenze parziali di parole

per simulare questo comportamento è necessario sommare più ranking function una phrase query e una allwords query se usate in una funzione rank danno lo stesso punteggio se si tratta di un match di frase, mentre se non vi è l'ordine la phrase query da 0

un'altra limitazione è l'impossibilità di filtrare con criterio match tot parole, che però è possibile reimplementarlo tramite operazioni sugli array

estraiamo i lessemi del testo della query, lo fa dentro Postgres altrimenti vi è il rischio di stemming incoerente se fatto con un tool esterno

```

1
2  WITH query_lexemes AS (
3    SELECT arr, cardinality(arr) AS n,
4          LEAST(
5            CASE
6              WHEN cardinality(arr) > 9 THEN
6                GREATEST(CEIL(cardinality(arr) * 0.5)::int, 1)
7              WHEN cardinality(arr) > 4 THEN
7                GREATEST(CEIL(cardinality(arr) * 0.6)::int, 1)
8              WHEN cardinality(arr) > 2 THEN
8                GREATEST(CEIL(cardinality(arr) * 0.9)::int, 1)
9              ELSE cardinality(arr)
10             END,
11            15) AS k
12   FROM (
13     SELECT array_agg(lex ORDER BY lex) AS arr
14     FROM unnest(tsvector_to_array(to_tsvector('italian',
15        'problema di accesso al portale utente')) AS lex
16   ) AS lexemes
17 ),
```

6 Descrizione del Lavoro Svolto

esegue la ricerca

```
1
2
3 candidates AS MATERIALIZED (
4     SELECT
5         "ticket_id",
6         "chunk_text" AS matched_text,
7         "chunk_counter",
8         (
9             ts_rank_cd("tsv_lang", to_tsquery('italian',
10             (plainto_tsquery('italian', 'problema di accesso al
11             portale utente'))::text || '.*'))
12             + ts_rank_cd("tsv_lang", to_tsquery('italian',
13             (phraseto_tsquery('italian', 'problema di accesso al
14             portale utente'))::text || '.*'))
15             + ts_rank_cd("tsv_lang", to_tsquery('italian',
16             replace(plainto_tsquery('italian', 'problema di accesso
17             al portale utente'))::text, ' & ', ' | ')))
18         ) AS raw_score,
19         cardinality(ARRAY(
20             SELECT unnest(tsvector_to_array("tsv_lang"))
21             INTERSECT
22             SELECT unnest(arr) FROM query_lexemes
23         )) AS matched_count,
24         (SELECT k FROM query_lexemes) AS required_k
25     FROM public.ticket_chunks
26     WHERE "field_name" = 'description'
27         AND tsvector_to_array("tsv_lang") && (SELECT arr[1 :
28         GREATEST(n - k + 1, 1)] FROM query_lexemes)
29         AND "chunk_language" = 'it'
30 )
```


6 Descrizione del Lavoro Svolto

```
1
2
3 SELECT
4     "ticket_id",
5     'description' AS "field_name",
6     matched_text,
7     "chunk_counter",
8     raw_score,
9     1.0::float8 AS field_weight
10 FROM candidates
11 WHERE raw_score >= 0.0
12     AND matched_count >= required_k;
13
14
```

non essendo funzionalità di ricerca full text non è possibile usarla nella funzione di ranking, nel config è possibile configurare le soglie e le percentuali,

è comunque utilizzabile nella configurazione una tsquery, solo che una anyword query su query grandi e basi di dati ampie non limita significativamente la base di dati

la ricerca su lingua non nota esegue questa esegue una tre volte la ricerca, una volta sul vettore tsv_simple, 1 svolta su tsv_lang where lang = it e una volta where lang = en scorre la base di dati 2 volte e poi i risultati vengono nuovamente combinati con union all

6.3.1 ricerca ibrida

le query costuite precedentemente non vengono eseguite subito ma vengono prima create tramite classi helper e poi eseguite.

questo permette di realizzare la ricerca ibrida come rrf che prende i 2 frammenti di query ricevuti e applica l'rrf lato db, tramite cte vengono materializzate e la posizione viene materializzata usando RANK OVER

6.4 Progettazione della ricerca linked

vengono generati gli sql frammenti di tutte le entità poi in base alla linked search configuration vengono stabiliti i join per espandere le tabelle poi union all e rescoring

rimane sempre solo una chiamata sql

6 Descrizione del Lavoro Svolto

6.5 retriever trial

6.5.1 esecuzione query

6.5.2 logging risultati

6.5.3 Dashboard

Capitolo 7

Conclusioni

In questo capitolo traggio le conclusioni sul progetto.

7.1 Consuntivo finale

Una volta terminato il progetto ho redatto il consuntivo orario finale nella Tabella 5 che suddivide in maniera approssimata le ore dedicate alle varie fasi.

Fase	Ore
<i>Onboarding</i> del progetto	5
Analisi dei requisiti	30
...	...
Totale	320

Tabella 5: Consuntivo orario finale.

7.2 Raggiungimento degli obiettivi

7.3 Requisiti soddisfatti

Arrivato alla fine del progetto ho implementato...

)

7 Conclusioni

7.4 Rischi occorsi e mitigati

I rischi emersi durante lo stage sono riportati in Tabella 6.

Descrizione	Mitigazione
R1 – Descrizione del rischio	Soluzione

Tabella 6: Rischi occorsi con la loro mitigazione.

7.5 Valutazione personale

Glossario

Elasticsearch: Motore di analisi e ricerca distribuita. Funge da piattaforma di retrieval e salva dati strutturati, non strutturati e dati vettoriali. 5

IR – Information retrieval: Insieme delle tecniche utilizzate per gestire la rappresentazione, la memorizzazione, l'organizzazione e l'accesso ad oggetti contenenti informazioni 56

Linked search: Modalità di ricerca che analizza ogni entità del database, con la possibilità di applicare un filtro, e combina i risultati secondo una configurazione che specifica come eseguire i join, vi è anche la possibilità di applicare un filtro al termine del join

LLM – Large Language Model: Modello di intelligenza artificiale addestrato su enormi quantità di testo per comprendere e generare linguaggio naturale, come GPT, Gemini o Mistral. Viene impiegato in compiti di analisi, estrazione di informazioni e generazione testuale. 5

Pgvectors: Estensione per Postgres, un database management system, che semplifica l'utilizzo dei vettori, consentendoti di archivarli, cercarli e indicizzarli direttamente nel tuo database relazionale. 5

RAG – Retrieval Augmented generation: Tecnica che permette ai LLM di reperire e incorporare informazioni da fonti di dati esterne.

Questo permette di recuperare informazioni rilevanti da database, documenti caricati, fonti web, ecc...

Il vantaggio principale è l'attualità delle risposte fornite dall'LLM e la possibilità di non dover usare documenti privati in fase di addestramento. 5

ranking – Ranking: Processo algoritmico con cui un sistema assegna un punteggio di rilevanza a un insieme di documenti rispetto a una query, ordinandoli in una lista decrescente. 57

similarity-search – Ricerca per similarità: Una funzione che cerca gli elementi della collezione più simili alla query secondo una misura di similarità e restituisce un ranking ordinato per grado di somiglianza decrescente. 57

Bibliografia